

Risk-Bounded Runtime Assurance for Dynamic Multi-Agent Systems under Partial Observation

AI Safety: Verifiable Identity and Evidence for Bounded Cross-Organisation Autonomy

Carsten Stöcker

28 September 2026



Except where otherwise noted, this work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). © 2026 Dr. Carsten Stöcker, Spherity GmbH.

Abstract

Applied AI safety requires a decision procedure between an agent’s proposal and its external effects. We construct a runtime supervisor that admits actions only when current mandate and assurance conditions both hold. For a bounded, partially observed multi-agent system, a finite model bank supports action-bound assurance leases. Joint lifecycle transitions retain shared dependencies, evidence and unfinished effects when workers join or retire. Each lease covers operation, delayed containment and pending exposure; a persistent episode account permanently debits its full continuation bound before execution. Under model containment, sound inference, mediated execution and a declared restriction on proposer information, we prove that adaptive renewal preserves a first-harm probability bound of epsilon plus calibration error. A separate refinement argument preserves authority and resource invariants. Exhaustive enumeration of 432 synthetic scenarios covers 120,060 terminal observation histories. In these tested settings, supported-model policies remain below the five-percent episode threshold, with maximum computed harm probability 3.73 percent. Posterior reuse preserves full-history decisions while reducing repeated inference. Eighteen calculations examine early renewal with inherited pending effects; finite controller checks reach 24,448 states and expose twelve control-omission counterexamples. Architectural extensions address cross-organisation contracts, verifiable identity and evidence, micro-to-macro supervision and temporary agents. Worked deployment cases cover frontier providers, independently operated models and cloud-edge systems, including testing, evaluation, verification and validation (TEVV) assertions. A dedicated kill-switch protocol distinguishes blocking new effects from verified containment and controlled restart. These extensions specify implementation and validation obligations. The contribution is a reproducible conditional construction for applied AI safety; it does not establish field calibration, protection against unmodelled dynamics or general AI alignment.

Keywords: runtime assurance; partial observation; multi-agent systems; risk budgets; dynamic membership; cross-organisation assurance; zero trust; applied AI safety; digital identity; business wallets; verifiable credentials; delegated authorisation; verifiable evidence graphs; trusted AI; kill switch

Contents

1	Introduction	4
2	Related work and position of the three paper programme	5
2.1	Established methods and the remaining question	5
2.2	Generator independent assurance and observation design for eligible actions	6
2.3	Control and data planes meet at the action decision	7
3	Running example and the meaning of a next action	9
3.1	Exact actions and exposure episodes	9
3.2	Six episodes that test the interface	10
3.3	A simple duration calculation	10
4	Contract, hidden state and assumptions	11
5	Calibration and posterior coverage	13
5.1	A finite-bank calibration method	13
5.2	Full posterior update with unobserved harm	14
6	Membership transfer with evidence retention	14
6.1	Scope-preserving lift	15
6.2	Concrete worker join	16
6.3	Retirement and pending work	16
6.4	Genuine new dependencies	16
7	Lease value through delayed containment	18
8	Conservative episode accounting under renewal	19
9	Eligibility and controller refinement	22
10	Finite-bank algorithm and computational cost	23
11	Counterexamples and information limits	24
11.1	Omitting containment delay	24
11.2	A fresh account at renewal	24
11.3	Duplicated common evidence	24
11.4	Discarding a retired worker's queue	25
11.5	Conditioning on a silent alarm	25
11.6	Observation indistinguishability	25
11.7	Candidate-specific error reset	25
11.8	Undisclosed proposer information	25
11.9	What additional monitoring can buy	25
12	Reproducible synthetic evaluation	26
12.1	Model and event order	26
12.2	Matched policies and metrics	27
12.3	Main results and the value of reuse	27
12.4	Early renewal and inherited pending exposure	30
12.5	Calibration and deliberate model failure	30
12.6	Numerical and empirical limits	31
13	Finite execution and accounting checks	31
13.1	Controller abstraction	31
13.2	Exhaustive results	31
14	Assurance across organisational boundaries	32
14.1	The unit of assurance is the joint episode	32
14.2	One joint action record and compatible continuation witnesses	33
14.3	A bilateral data release example	34
14.4	Architecture patterns and bounded swarms	34
14.5	Contracts and reciprocal assertions between independent systems	37
14.6	Identity delegation and verifiable evidence graphs	38
14.7	Architectural levels and placement of wallets and decision modules	39
14.8	Dynamic agent creation and task scoped lifecycles	42

14.9 Operational alignment and the claim that can be verified	43
15 Deployment patterns operational oversight and verified containment	44
15.1 Deployment taxonomy and governance responsibilities	44
15.2 Worked case of a frontier provider and its customers	45
15.3 Closed models and verifiable assurance assertions	47
15.4 Independently operated open models	48
15.5 Hybrid cloud and edge execution	49
15.6 Internal monitoring and independent oversight	49
15.7 Kill switch for verified containment and controlled restart	50
15.8 Deployment evaluation and intervention tests	51
16 Applied AI safety and implementation obligations	52
16.1 A practical prototype boundary	52
16.2 Evaluation and incentives	53
16.3 Contributions and limits	54
A Reproducibility and research status	55
References	56

1 Introduction

An AI agent can be authorised to act while lacking adequate evidence that its next action meets a declared risk constraint. A valid credential identifies a claim and its issuer. It does not establish the current integrity of a service, the quality of a sensor, or the time needed to stop a queued operation. Applied AI safety must connect these questions at the point where a proposal becomes an effect.

Our running example concerns two engineering organisations. Their agents book shared compute capacity and subsequently release protected data through a controlled gateway. Paper 1 analyses the booking as an atomic digital commitment with a shared monetary budget and revocable authority [1]. This paper adds a separate release decision. The named harm is the first release of protected data outside a declared isolation boundary. A successful booking neither performs that release nor certifies the booked service’s isolation.

The central decision is

$$\mathcal{A}_t^{\text{eligible}} = \mathcal{A}_t^{\text{mandate}} \cap \mathcal{A}_t^{\text{assurance}}.$$

The mandate set expresses what the represented principal currently permits. The assurance set expresses which exact actions have a supported continuation under the declared harm limits. A risk threshold restricts the latter set. It never creates missing authority. Both sets may change between reservation and execution.

This becomes difficult when workers change, observations are incomplete and interventions take time. A new worker may share the same weak provider as existing workers. Retirement may leave a remote release queued. A quiet monitor may have missed harm that already occurred. Repeated renewals can spend the same apparent risk allowance many times unless the account persists. These are different mechanisms. A useful analysis must represent each one explicitly.

We give a restricted construction that combines four requirements: preserve a joint belief through covered membership changes; calculate risk through the end of containment and pending exposure; charge successive policies to one exposure episode; and enforce the resulting decision against current authority. Its mathematical foundations are established. The contribution is their explicit connection through a lifecycle and execution contract, with a proof, reproducible calculations and failure examples. We do not claim that filtering, dynamic risk budgets, runtime switching or an additional controller guard are new in themselves.

The core probability result applies to a finite bank containing the true complete model. Calibration can remove models with a stated coverage error. Decisions use the supervisor’s declared history, and extra proposer randomness must be independent of hidden state given that history. A proposer using undisclosed telemetry requires a different selection model or a stronger assurance construction. This restriction is essential and receives its own counterexample.

Organisational boundaries add separate authority policies, private observations and independently operated gateways. The running example already involves two principals, but the proof assumes one coherent logical supervisor and episode account. Section 14 explains when a distributed implementation can preserve that abstraction, and why independent local approvals are insufficient for joint safety. Sections 14.5–14.8 develop reciprocal eligibility assertions, identity and delegation bindings, evidence graphs and the placement of wallets and protected decision services. They distinguish instance, task and governance scopes, and specify how temporary workers enter and leave a covered episode. The resulting alignment claim concerns conformance to the agreed operational contract.

Practical deployment adds a further question: which party can observe and control each effect? A frontier provider may operate the model while its customer controls tools and data. An independent operator can inspect a local model yet still leave an unmediated execution path. Section 15 develops worked deployment cases, including assertions about closed-model TEVV and safety mechanisms. It connects internal monitoring and external oversight to verified containment, with reference to the NIST AI Risk Management Framework (AI RMF) and the UK AI Security Institute (AISI). Section 15.7 specifies the kill switch as a protected containment and restart protocol. Section 16 sets out staged implementation and validation obligations. These cases retain the proof’s conditions and distinguish model-level evaluation from evidence about a deployed continuation.

The continuous-time hazard example in Section 3 is a teaching baseline. The proved construction uses finite decision ticks because these make membership, event ordering and pending effects explicit. The experiments integrate constant within-segment hazards into finite transition probabilities.

The results have three kinds of evidence. Analytical arguments establish conditional statements for the declared model class. Numerical enumeration checks all observation histories in small synthetic instances. A separate breadth-first exploration checks finite controller interleavings. These forms of evidence support different claims; none constitutes deployment validation of an LLM agent or a physical plant.

Reader guide. Sections 4–10 contain the finite-model construction and its proof obligations. Section 11 isolates the main failure mechanisms. Sections 12–13 distinguish numerical enumeration from finite controller checking. Sections 14–16 translate the construction into proposed organisational and deployment interfaces. These interfaces motivate Paper 3, while the probability theorem here remains independent of a private-proof implementation.

2 Related work and position of the three paper programme

2.1 Established methods and the remaining question

Chance-constrained planning with partially observable Markov decision processes (POMDPs) is established, including Risk-bounded AO* (RAO*), an AND–OR graph search algorithm [2]. Simplex-style runtime assurance separates an advanced controller from a protected switching and recovery mechanism [3]. Persistent risk allocation is also established: Huang et al. study planning with a dynamic risk budget [4]. Conditional safety certificates for open adaptive systems address runtime composition [5], while runtime-assurance verification for autonomous aircraft includes recovery and timing obligations [6]. These works rule out broad claims that this paper introduces adaptive certification, risk accounting or delayed recovery.

Interval-POMDP shielding constructs conservative action decisions under imperfect perception [7]. Local shields for decentralised POMDPs address safety when agents have different observations [8]. Confidence sequences provide time-uniform statistical statements under their sampling assumptions [9], and verified hidden-Markov monitors offer another related inference route [10]. The present baseline uses independently labelled offline calibration and exact finite-model filtering. It does not apply a confidence sequence directly to an unobserved hazard rate.

Recent multi-agent risk allocation already couples agents through a shared constraint [11]. Our question is narrower: which previously supported information may be carried into a changed membership state, how must unfinished effects be retained, and how does a replacement policy remain attached to one episode account? The answer below is a sufficient scoped transfer rule

and a deliberately conservative composition rule. It is not a general efficient solution to partially observed multi-agent planning.

2.2 Generator independent assurance and observation design for eligible actions

In *Generator-Independent Runtime Assurance under Partial Observation*, Wan et al. [12] characterise admission soundness through simultaneous setwise soundness: the whole admitted set excludes nonviable proposals. With past decisions fixed, setwise failure probability equals worst-case nonviable-admission probability over selectors observing latent state, realised noise and the certified set, under no-bypass and measurable-selection assumptions. Contract safety additionally requires a design-time certificate for viable executions and fallback. It is a sufficient consequence, not the equivalence itself. This distinguishes checking the selected candidate from supporting the complete set available to a changing generator.

Our construction uses a narrower information interface. A containing finite model bank bounds complete candidate policies after conditioning on the supervisor’s declared history. Privileged telemetry must enter that history or the selection likelihood; otherwise candidate choice can invalidate the posterior. Our theorem therefore does not cover an arbitrarily state-informed proposer. Its $\varepsilon + \delta$ bound accounts for accumulated first-harm continuation risk and model-exclusion error under Section 8’s assumptions. The operational connection is mediation of an eligible action together with its monitoring, stopping and pending effects. Eligibility remains a conditional risk claim, rather than a guarantee of harmlessness in every hidden state.

Observation Design for Certified Control Authority: Projection–Estimability Separation and Active-Face Equivalence [13] studies linear sensing with independent Gaussian noise and constraints affine throughout the confidence region. For information matrix M and constraint normal c , distinguishing two states along c requires $c^\top M c > 0$; finite robust uncertainty over the full ellipsoidal region requires $c \in \text{Range}(M)$. A partly measured direction can support discrimination while an unrestricted null component blocks certification. Thus detecting a difference and bounding the quantity needed for admission are separate tasks. For finite nonzero constraint normals and compact probes with continuous information maps, an attained optimum positive definite on the recoverable quotient satisfies an active-face condition. Constraints limiting certified slack define an L -optimal target, weighted by active multipliers and inverse remaining slack; one binding face recovers c -optimal design. These are design-time statements. For a reachability-truncated region, its actual support is needed for an exact test; the full-ellipsoid penalty remains conservative. Both studies are preprints.

For our programme, this motivates acquiring evidence that can change a binding assurance constraint, accounting for its cost, delay and effects. A service-identity check may establish the correct recipient while an isolation probe addresses the condition that currently prevents release. Paper 2 compares fixed monitoring policies and a binary probe illustration; it does not implement the companion optimisation or import its Gaussian assumptions into the finite-bank proof.

Paper 1 supplies mandate, resource and execution constraints, including costs and revocation [1]. Paper 2 supplies the conditional continuation decision. The companion Paper 3, *Private Verification of Action Eligibility across Trust Domains*, specifies private evaluation over bounded evidence views, typed cross-domain claims and protected effect admission [14]. Its conditional control arguments and finite checks do not establish a deployed private-proof system or transfer this paper’s probability bound automatically. Measurement validity, information scope, model coverage and effective containment retain their own proof obligations. The two preprints discussed above support distinct parts of our design: the admission interface and the choice of observations.

Our lifecycle transfer, permanent accounting and execution binding develop a specific operational construction; the synthetic results do not replicate or validate either preprint.

2.3 Control and data planes meet at the action decision

Software-defined networking (SDN) separates control from forwarding through defined interfaces [15]. Zero Trust Architecture (ZTA) separates policy decision, administration and enforcement while using contextual information [16]. Eclipse Dataspace Components likewise separates negotiation and policy processing from data movement [17]. We use “convergence” to mean that policy, current evidence and actual execution participate in one feedback cycle. It does not mean collapsing protected components into one plane or granting the proposer control of enforcement.

In the example, the control side supplies the principal’s release mandate and the current assurance lease. The transfer side observes the actual service and mediates data movement. Telemetry updates the next decision; the gateway applies the resulting restriction to the next bounded transfer. A changed provider, missing probe or stale lease must therefore affect executable scope, not merely an audit dashboard.

This matters for current cyber-physical systems because computation can trigger effects whose timing and recovery are constrained. Operational technology (OT) security already recognises performance, reliability and physical safety as relevant design requirements [18]. A stop request in a battery installation or robot cell is not an instantaneous safe state. Such applications require local protective controllers and their own hazard models. They are motivating analogies here, not validated applications of the data-release experiment.

Agentic systems increase the frequency of these decisions. An agent can create workers, choose tools, retry requests and change dependencies during one task. Those changes can alter both authority and exposure. The engineering hypothesis is that useful autonomy will require frequent, explicit evaluation of the mandate–assurance intersection. The evidence in this paper supports a bounded mechanism for doing so; it does not establish universal adoption or a general scaling result.

Authority and assurance meet at execution

$$\mathcal{A}_t^{\text{eligible}} = \mathcal{A}_t^{\text{mandate}} \cap \mathcal{A}_t^{\text{assurance}}$$

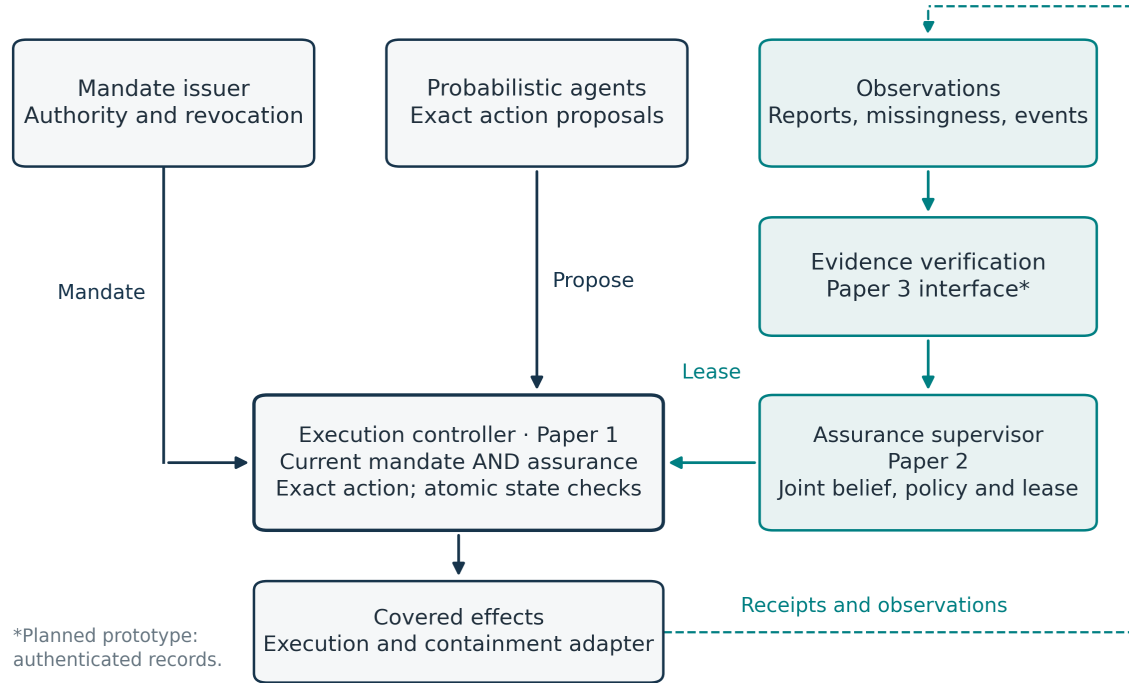


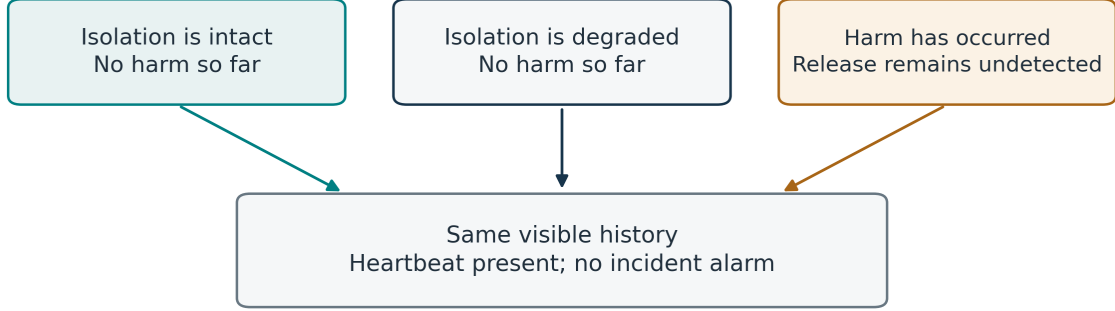
Figure 1: Authority, assurance and execution in the three-paper programme. The protected gateway combines current mandate checks with an action-bound assurance decision. Paper 3 specifies the proposed private-proof and claim-validation interface [14]. A proof about supplied evidence does not establish the hidden state.

3 Running example and the meaning of a next action

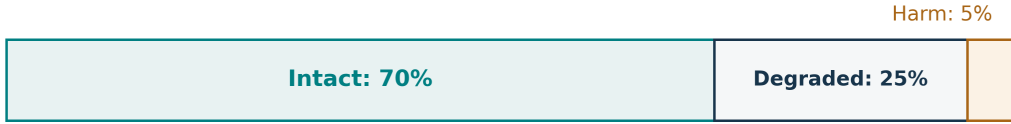
The hidden state and the belief are different

One state is realised; several states may fit the same observed history.

A Hidden alternatives



B An illustrative belief over the joint state



Illustrative probabilities. The absorbing harm flag remains in the belief.

Figure 2: Hidden operating conditions and the supervisor’s belief are different objects. The illustrative distribution retains probability for undetected prior harm. A signed or quiet report does not reveal the hidden state.

3.1 Exact actions and exposure episodes

A **next action** is the smallest externally relevant operation, or bounded operating segment, to which the controller can attach an enforceable decision. Its record identifies the represented principal, running instance, operation, target, payload, time window, relevant dependencies and continuation. “Complete the engineering project” is too broad. “Release chunk 17 of dataset D to endpoint E under isolation configuration c” can be checked and mediated.

Table 1 gives examples. An operating segment can contain several permitted calls only if the assurance calculation covers their joint policy. Separately assessing each call does not establish a bound for arbitrary combinations.

Table 1: Concrete next-action decisions in the running example.

Next action	Mandate question	Assurance question
Reserve 60 compute credits	Is this instance authorised and is the shared budget sufficient?	Does this action itself create a covered exposure, or only a booking record?
Release one protected data chunk	Are target, dataset and purpose in scope?	Does current isolation evidence support the release and its pending effects?

Table 1 (continued)

Next action	Mandate question	Assurance question
Start a bounded remote job	Is the exact workload permitted?	Are queueing, execution and containment within the supported policy?
Admit another worker	Is delegation allowed under the root mandate?	Does its joint dependency model preserve evidence and account for new exposure?
Renew transfer for a bounded interval	Does authority remain current?	Does the complete continuation fit the remaining episode allocation?
Run an isolation probe	Is the measurement operation permitted?	Is the probe informative, and are its own effects covered?

An **exposure episode** starts from a declared initial state and ends only when its covered exposure closes. Renewal, retry, a new session or a new worker identifier does not reset that episode. The principal owns both a monetary mandate and a separate risk allocation. Credits and first-harm probability are different quantities. One cannot be traded for the other without an additional declared decision rule.

Episode scope is fixed by the declared task and exposure contract. A temporary interval with no active or pending work can close one operating segment while the same task still permits later segments. Such an interval does not reset its account. Final episode closure requires completion of the declared task exposure or an explicitly charged residual-tail argument; a restart under another instance or session remains subject to the original scope rule.

3.2 Six episodes that test the interface

A worker first presents a valid credential but its provider probe is missing. Authority can remain valid while release assurance becomes insufficient. It may still be permitted to request a probe or choose a covered restricted action.

A second worker receives less informative observations. Under a containing model set, the supported lease can become shorter or unavailable. A missing report is evidence whose likelihood must be modelled; it is not automatically neutral.

A third worker receives a stop command after a remote job has accepted a release. The gateway can prohibit new dispatches while the accepted effect remains unresolved. The episode continues through that exposure.

A fourth worker joins on the same provider. It adds capacity but does not supply an independent copy of provider evidence. A fifth worker runs on a compromised service that emits exactly the same reports as a healthy one. No amount of repeating those reports resolves that distinction. A sixth worker retries or forks after refusal. Its attempts remain under the same declared evidence scope and episode account.

3.3 A simple duration calculation

Suppose a teaching model has constant upper hazard rate λ per minute. Operation lasts h minutes, and exposure continues for d minutes after a stop request. The first-harm probability is

$$r(h, d) = 1 - \exp[-\lambda(h + d)].$$

For allowance ε , a feasible operating duration satisfies

$$h \leq \frac{-\log(1 - \varepsilon)}{\lambda} - d.$$

At $\varepsilon = 0.01$ and $d = 2$, the limit is 8.05 minutes for $\lambda = 0.001$, but 1.35 minutes for $\lambda = 0.003$. These rates are illustrative inputs, not estimated cyber incident rates. A negative right-hand side means even immediate stopping fails to meet this particular allowance under the assumed delay. Setting the permitted duration to zero must not be reported as a successful certificate in that case.

A general score from a classifier may help prioritise investigation. It is not a probability bound unless its meaning, calibration, horizon and selection conditions support that interpretation. With several harms, the baseline uses separate constraints. It does not average an unacceptable privacy risk against a low financial risk.

Containment delay consumes the exposure allowance

Illustrative constant hazard; 1% allocation; containment effective after 2 minutes.

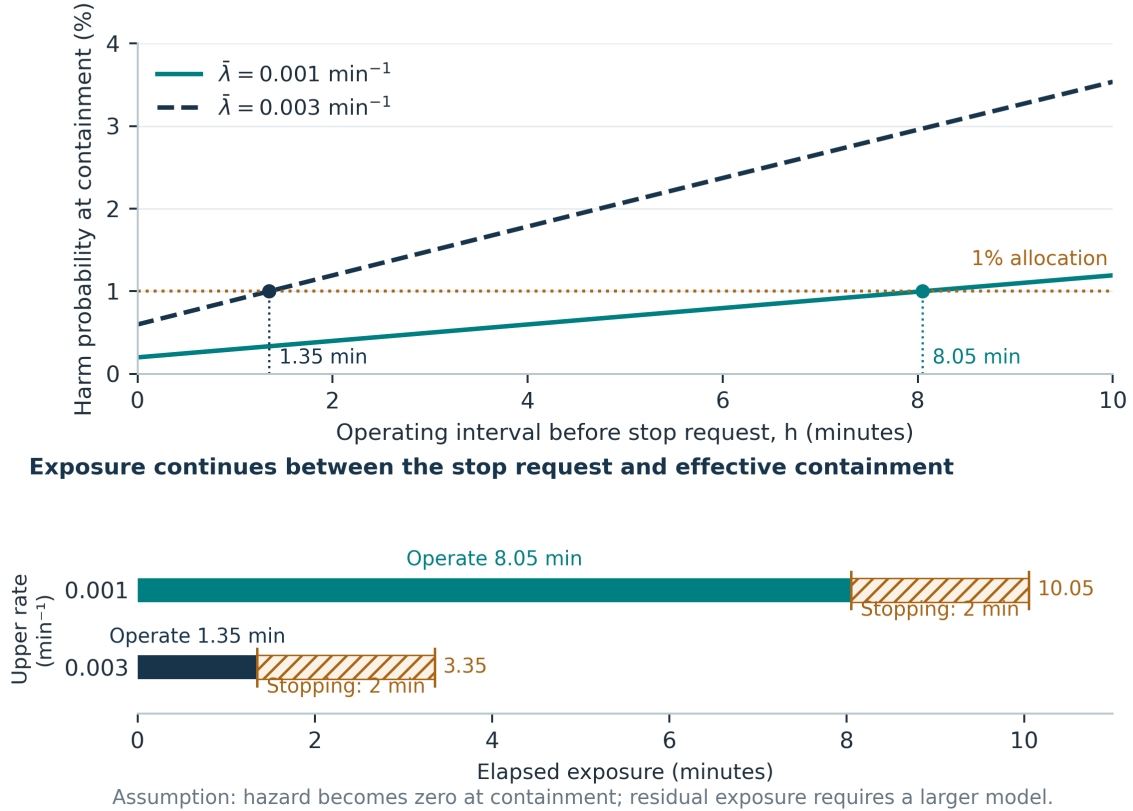


Figure 3: Illustrative lease duration with delayed containment. The one-percent threshold includes the two-minute interval after stopping is requested. The two rates are teaching assumptions. They are unrelated to the synthetic benchmark’s five-percent episode threshold.

4 Contract, hidden state and assumptions

One **episode** has one named harm: first release of protected data outside a declared isolation boundary. It starts with a stated initial distribution and continues through all renewals, retries,

worker changes and pending effects until its declared exposure closes. A different business transaction or agent identifier does not automatically create a new episode. Other harms require their own definitions and constraints.

Use discrete decision ticks for the formal baseline. A state includes

$$X = (Z, W_1, \dots, W_m, Q, C, H).$$

Here Z describes common dependencies such as provider isolation; W_i is a worker’s relevant local state; Q retains in-flight effects, including those of retired workers; C is the known operating mode and containment countdown; and $H \in \{0, 1\}$ records whether the named harm has occurred. The flag is absorbing, while the other fields may continue to change after harm. The supervisor need not observe H . Membership and time indices can change the finite state space. Bound the number of workers, queue size and episode decision opportunities.

An exact action includes its principal, instance, operation, target, payload, duration, policy and relevant dependencies. An operating segment is an exact bounded action envelope plus an enforced continuation policy. All workers and pending effects governed by the episode are included in the **global** envelope. Calculating each worker’s isolated risk and silently ignoring interactions is outside the construction.

Organisations are represented through their principals, workers, obligations and effects within this joint state. Their number does not alter the theorem, but separate private histories or asynchronously installed policies can violate its assumptions. A physically distributed supervisor needs a refinement preserving the declared decision history, continuation and account semantics. Section 14 states this application boundary; no distributed protocol is proved here.

The theorem assumptions are:

1. **Finite model containment.** A finite bank Θ_0 of complete models contains the fixed true model θ_* . Each model supplies initial conditions, ordinary transition/observation kernels, all admitted lifecycle kernels, action effects, containment and pending-work semantics. The bank is a declared modelling assumption; its discretisation does not cover omitted intermediate or adversarial dynamics automatically.
2. **Independent calibration.** Offline labelled calibration data D are independent of the exogenous deployment randomness and initial state for fixed θ_* . The selected policy may depend on D ; the declared deployment kernels still apply after fixing it. A retained bank $\Theta(D)$ obeys $\Pr_{\theta_*}\{\theta_* \notin \Theta(D)\} \leq \delta$. Alternatively, use all of a known containing bank and set $\delta = 0$. The latter is a structural robustness result, not an empirical calibration claim.
3. **Exact or outward-bounded inference and reachability.** The retained posterior sets and risk calculations contain the true-model quantities. Do not prune a positive-likelihood model merely because its probability is numerically small. An implementation using floating-point arithmetic must state its numerical status separately from this ideal result.
4. **Mediated execution and information scope.** The controller implements the exact admitted policy until the next valid replacement. All decision-relevant observations of the selector are represented in the declared history; any additional private randomisation is conditionally independent of unobserved deployment state and future exogenous deployment noise, given that history. Decisions are nonanticipating. A proposal informed by undisclosed private telemetry is outside this baseline unless its informative selection likelihood is modelled. A mandatory fallback/containment continuation executes on expiry, failed renewal or unavailable assurance. The model covers authority revocation and other early stop triggers. Missing an observation is itself an observation symbol with its declared likelihood, not automatically an uninformative event.

5. **Lifecycle coverage.** Each admitted join, fork, retirement or dependency change uses a covered joint transition. Controlled joins occur at a controller boundary before new dependent effects. Asynchronous covered changes must already be included in the old policy’s dynamics through detection and reaction. Transfers retain common causes and pending effects and account for harm produced by the transition itself. An unexpected change outside the bank invalidates the quantitative guarantee and triggers the declared response; it is not hidden inside δ .

6. **Episode discipline.** The risk account persists across renewals and identities, has an authorised owner and threshold, and receives a debit before the new policy produces an effect. The initial harm flag is known false, or its initial probability is charged explicitly. There is no uncharged exposure before the first lease, between charged segments, or after final closure. If the initial condition already carries autonomous pending exposure, a charged initial protective continuation must cover it; merely refusing the first proposed action is insufficient.

7. **Protected state.** Authority, accounting, configuration, membership and assurance checks use coherent state at commit. This imports Paper 1’s stated execution assumptions, not an unproved assertion about ordinary remote APIs.

The first implementation uses a fixed, finite policy library. Each policy specifies its initial exact action, monitoring, permitted segment, observation-dependent early stopping where applicable, expiry handling, and containment. A proposer satisfying the declared information restriction may suggest any candidate, but the controller only executes the selected certified policy. A uniform guarantee for arbitrary choices within a broad envelope requires a separate worst-case reachability calculation over all allowed selectors. In particular, this result does not claim soundness for a selector with arbitrary privileged knowledge of hidden state. Duration-policy pairs are checked by finite enumeration; monotonicity of risk in duration is not assumed for policies whose stopping behaviour changes.

5 Calibration and posterior coverage

5.1 A finite-bank calibration method

For $0 < \delta < 1$, suppose each of M declared transition or observation rows is estimated using $N_r \geq 1$ independent labelled draws, with k_r outcome categories and empirical probabilities $\hat{p}_{rj}$. Define

$$L = \sum_{r=1}^M k_r, \quad w_r = \sqrt{\frac{\log(2L/\delta)}{2N_r}}.$$

Keep every complete model for which

$$\left| p_{rj}^\theta - \hat{p}_{rj} \right| \leq w_r \quad \text{for all calibrated } (r, j).$$

Hoeffding’s inequality [19] followed by a union bound gives simultaneous coverage at least $1 - \delta$, provided the true complete model is in Θ_0 . Different rows need not be mutually independent for this union bound. The draws used to estimate each row must satisfy the stated sampling condition. Bonferroni-adjusted exact binomial intervals are an alternative for binary rows. Rows without calibration remain unconstrained within the original bank. Unlabelled deployment observations are used for filtering, not treated as new labelled calibration samples.

This construction is elementary and deliberately limited. It does not claim to identify hidden hazard rates from uninformative telemetry. An empty bank means that the chosen model/calibration

procedure supplies no certificate; it is not permission to enlarge the interval opportunistically. An independently specified protective procedure still has to execute.

The key advantage of calibrating complete models jointly is **selection validity**. Once the true model is retained, every exact query evaluated under it is covered, including adaptively chosen actions, durations, histories and admitted membership transitions. No additional per-query error is introduced by querying that same retained bank. This claim relies on sound evaluation and on the scope of the complete bank; it does not protect an additional learned or approximate candidate-specific classifier automatically.

5.2 Full posterior update with unobserved harm

For each retained model, let $b_t^\theta(x) = \Pr_\theta(X_t = x \mid \mathcal{I}_t)$, where $\mathcal{I}_t$ is the visible deployment history including the fixed calibration record, policy choices and account state. Let $K_\theta^u(x', y \mid x)$ be the joint transition-observation kernel for executed action u . Then

$$b_{t+1}^\theta(x') = \frac{\sum_x b_t^\theta(x) K_\theta^u(x', y_{t+1} \mid x)}{\sum_{x, x'} b_t^\theta(x) K_\theta^u(x', y_{t+1} \mid x)}. \quad (1)$$

Zero-likelihood histories can eliminate an incompatible model exactly. Positive but small likelihoods cannot be set to zero without a justified enclosure. An observable stopping time also carries information; the recorded controls and observations must permit the filter to reconstruct that information.

The filter includes $b_t^\theta(H = 1)$. A quiet monitor does not make this mass zero. The risk of *new first harm* after time t is calculated from the whole posterior without conditioning the visible history on an unobserved survival event. The ordering is: record the actual observation or absence; update the joint posterior; evaluate recorded proposals; choose and charge a policy; execute its action transition; record later observations. A proposal carrying additional information requires a corresponding observation/selection likelihood before evaluation. Conditioning on controls as if they were independent interventions is justified by the shared-history information restriction, not by complete mediation alone.

Lemma 1 (history-wise posterior coverage). Fix calibration data D with $\theta_* \in \Theta(D)$. Under the model and inference assumptions, the retained family contains the true conditional state distribution at every reached history of positive true-model probability, including histories at adaptive renewal times.

Proof. The initial distribution is contained by assumption. Equation (1) is the conditional probability identity under the true kernel. Its denominator is positive at each reached history of positive probability. Covered lifecycle transitions are another such kernel. Induction over transitions proves the result. Choosing renewal times from the visible history does not change the conditional identity, since the policy and decisions are included in that history. No independence between deployment observations is required beyond the declared complete model. $\square$

Meaning. A noisy or missing probe can broaden the range of current hazard estimates. Repeating the same uninformative evidence never licenses setting an inconvenient hidden state to zero.

6 Membership transfer with evidence retention

Let e be a controlled lifecycle event with joint kernel $L_e^\theta(x^+ \mid x)$, and let the new state use an enlarged or reduced representation. Before any new observation,

$$b^{\theta,+}(x^+) = \sum_x b^{\theta,-}(x) L_e^\theta(x^+ | x). \quad (2)$$

If the occurrence of the lifecycle event is itself informative about hidden state, use a joint event-observation kernel and Bayes normalisation. Formula (2) alone is appropriate for an event chosen by the recorded controller, or a kernel already conditioned on all information conveyed by the event. Joining cannot be treated as uninformative merely for convenience.

6.1 Scope-preserving lift

Write the old state as (R, U) , where R contains fields to retain. A join kernel is **scope preserving for R** when

$$L_e^\theta(r', u' | r, u) = 0 \quad \text{whenever } r' \neq r. \quad (3)$$

The new U' can include a worker whose condition is correlated with the shared provider, parent, model and queue. For a harmless admission bookkeeping step, R includes Z, Q, H . A lifecycle action that itself releases data is not such a harmless lift: its first-harm probability must be included in the admitted action risk.

Theorem 1 (joint transfer and scoped evidence reuse). Under (2)–(3), the retained marginal is unchanged:

$$\sum_{u'} b^{\theta,+}(r, u') = \sum_u b^{\theta,-}(r, u). \quad (4)$$

Consequently, evidence supporting this marginal remains usable after the covered lifecycle event. If all future relevant transition-observation and harm laws depend on a full state only through R , and the same admitted policy chooses its controls from the retained observation history, then forward assurance calculations from the marginal on R equal calculations from the full transferred posterior. A policy that reads a discarded variable need not satisfy this condition. If the new membership changes the future relevant kernel, the marginal can still be retained, but reachability must be recomputed under the changed kernel. An old risk value is reusable only when its own dependency and kernel conditions are unchanged.

Proof. Sum (2) over u' . Condition (3) removes all terms with an old retained coordinate different from r . The row-stochastic property gives $\sum_{u'} L_e^\theta(r, u' | r, u) = 1$, yielding (4). For the second statement, the assumed controlled lumpability makes each one-step law of the retained state, observation and harm independent of the discarded coordinate. Repeated substitution proves equality of all finite observable path and first-harm probabilities. Changed kernels invalidate this second equality with the *old* future model, although they do not affect marginal identity (4). $\square$

Conditions for use. This is a sufficient marginal-reuse criterion, not permission to ignore arbitrary cross-agent dependence. Whether R is sufficient must be established for the action family and observation law. The proof is a standard pushforward and lumpability argument applied to lifecycle assurance. The paper’s proposed contribution is the explicit lifecycle contract and its integration with exposure accounting, not a new probability identity.

6.2 Concrete worker join

For $X^- = (Z, W_1, \dots, W_m, Q, C, H)$, append

$$W_{m+1} \sim L_e^\theta(\cdot \mid Z, W_1, \dots, W_m, Q, C, H).$$

The joint distribution of the old fields is preserved. A shared unhealthy provider does not become healthier because another worker joins. The new worker can alter the next-action hazard and work throughput, so the new risk is recomputed. Section 14.8 applies this join rule to task-scoped agent creation and activation. Signed evidence remains relevant to the same provider and version if its claim and dependency contract cover the event; the statistical posterior is not reset merely because a new instance identifier exists.

6.3 Retirement and pending work

Removing future permission for worker i does not remove an already queued disclosure. Transfer its in-flight effects to Q , retaining its local state if needed to predict the queue. Marginalising W_i is justified only if the resulting model is sufficient for later transition, observation and harm laws. Otherwise keep it as latent residual state until closure. A retired worker's pending effect can still produce the episode's first harm. The retirement protocol in Section 14.8 therefore retains responsibility for those effects after an instance is removed.

6.4 Genuine new dependencies

A new provider or sensor absent from the calibrated bank is not covered by this reuse theorem. A model extension must supply a containing joint kernel family and its coverage argument. A Cartesian uncertainty set can be conservative without assuming probabilistic independence, but the actual state couplings still require specification. Online bank replacement or recalibration requires a joint coverage argument for all accepted replacements; the coverage allowance cannot restart with each provider or instance. Theorem 2 uses one retained bank. The experiment in Section 12 stays within a predeclared lifecycle family.

A new worker can share an existing hidden cause

Membership changes the joint model; duplicate reports do not create independent evidence.

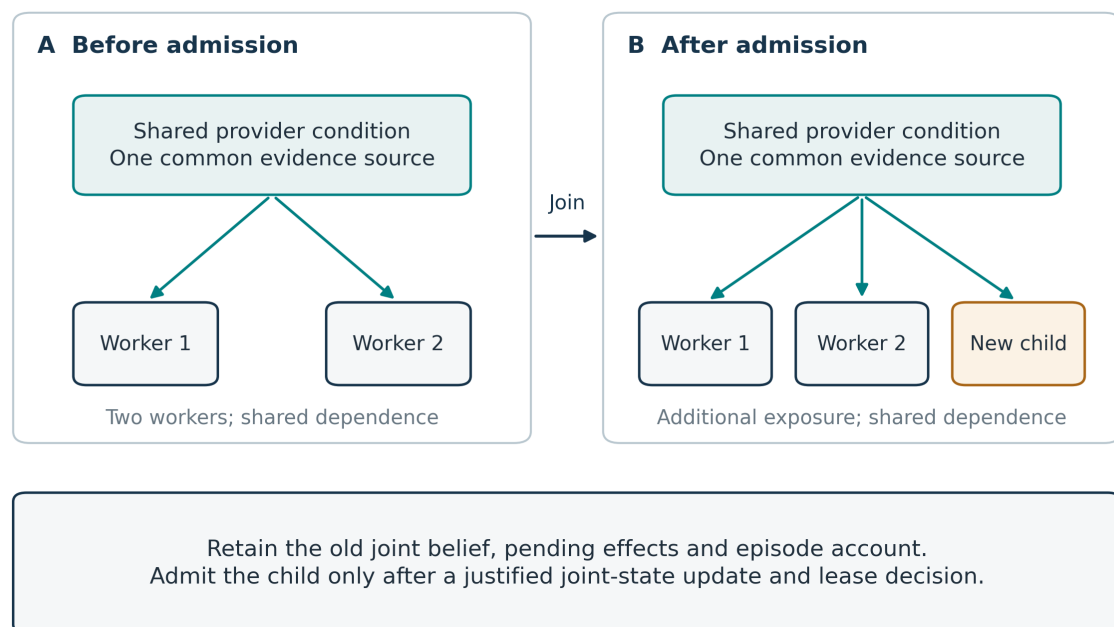


Figure 4: A covered worker join preserves the shared latent dependency and existing pending effects. New membership can change future risk and require a new calculation even when the retained marginal is unchanged.

7 Lease value through delayed containment

A candidate lease $\ell = (a, h, \pi)$ specifies an initial exact action a , up to h operating ticks, and a fully specified continuation π , including monitoring, early restriction and mandatory containment. Its terminal exposure time includes stop delivery, effective containment and all pending work. The closed state has zero further hazard for the declared event in the baseline. A nonzero residual tail requires an explicit bound and is part of the charged continuation.

For a candidate issued at t , define

$$\begin{aligned} r_t^\theta(\ell) &= \Pr_\theta^\ell(t < \tau_H \leq T_{\text{close}} \mid \mathcal{I}_t), \\ \bar{r}_t(\ell) &= \max_{\theta \in \Theta_t(D)} r_t^\theta(\ell). \end{aligned} \tag{5}$$

Here τ_H is the first-harm time, and Θ_t excludes only exact zero-likelihood hypotheses. Prior unobserved harm is excluded from the incremental event by its definition, but remains in b_t . Equation (5) does not replace its condition with $\tau_H > t$.

The decision time denotes the state immediately before the candidate's first effect transition. An instantaneous effect of that transition belongs to the new lease's charged segment. This convention gives the strict lower endpoint in (5) the same meaning as the segment boundary in Section 8.

For a fixed finite-horizon policy, standard dynamic programming computes this value. With $h(x)$ denoting the state's harm flag, set the terminal value to zero after verified closure. For any earlier policy history g ,

$$\begin{aligned} v_k^\theta(x, g) &= (1 - h(x)) \sum_{x', y} K_\theta^{\pi(g)}(x', y \mid x) \\ &\quad \cdot \left[h(x') + (1 - h(x')) v_{k+1}^\theta(x', gy) \right]. \end{aligned} \tag{6}$$

Then $r_t^\theta = b_t^\theta \cdot v_0^\theta$. Policy modes, countdowns and membership events can be compiled into the augmented finite state. If a validated tail bound $g_\theta(x)$ remains at the terminal horizon, set $v_T(x) = (1 - h(x)) g_\theta(x)$. A fixed matrix exponential is appropriate only for its declared continuous-time model and policy; it does not evaluate an arbitrary observation-dependent controller.

Lemma 2 (conservative lease reachability). Under Lemma 1 and the numerical/model assumptions, (5) bounds the true conditional incremental first-harm probability of executing the complete candidate continuation.

Proof. Equation (6) partitions the first transition into immediate first harm and continued unharmed operation. Backward induction establishes its value for each retained model. Maximisation includes the true model at each reached history. $\square$

Interpretation. An expiry record does not stop an already queued operation. The counted interval ends when the specified containment/queue procedure ends the relevant exposure, or when a separately bounded tail begins.

First-harm limitation. A first-harm metric saturates after the first occurrence. It does not bound the number or severity of later harms. The implementation should include independent incident-handling requirements and, where needed, additional harm-count or severity constraints. It must not present decreasing incremental first-harm probability after suspected harm as evidence of a healthy system.

8 Conservative episode accounting under renewal

Let $0 \leq \varepsilon \leq 1$ be the operational episode allowance. The following baseline deliberately charges forecast containment exposure again when a lease is renewed. It is conservative and avoids an unproved risk refund.

At issue time T_n , choose a nonnegative, $\mathcal{I}_{T_n}$ -measurable debit $\alpha_n \geq \bar{r}_{T_n}(\ell_n)$. Permanently update

$$B_{n+1} = B_n - \alpha_n, \quad B_0 = \varepsilon, \quad (7)$$

and issue only if $\alpha_n \leq B_n$. Never restore a debit merely because a report was quiet, the worker retired or the old fallback was superseded.

An accepted renewal can occur at a visible-history stopping time no later than the current policy's default closure. Until replacement, the old policy is followed exactly, including its expiry and stop logic. The next issued lease takes the full current posterior and outstanding queue as its initial state and covers the entire episode's continuing effects. On failed renewal, execute the current already covered continuation. A system must not wait in an unmodelled active state for the assurance solver to return.

Define the actual segment for lease n to start immediately before its charged initial action and end immediately before the next accepted lease's charged initial action, or at the final closure. Events caused by an instantaneous new membership/action transition belong to the new segment. This is the event-ordering convention used at the shared boundary. Distinct segments partition first-harm opportunities.

Theorem 2 (episode bound under adaptive renewal). Under Assumptions 1–7, Lemmas 1–2, the prefix replacement condition above, $H_0 = 0$, and pathwise $\sum_n \alpha_n \leq \varepsilon$,

$$\Pr_{\theta_*}(\tau_H \text{ lies in the episode}) \leq \varepsilon + \delta. \quad (8)$$

With initial harm probability bounded by β_0 , either reserve it from ε or add β_0 to the right-hand side. The statement extends to several declared harms by maintaining and proving the corresponding constraints separately. It does not justify replacing them with a weighted score.

Proof. First fix a realised calibration dataset D for which the true model is retained. Let E_n be the event of first harm in the actual segment assigned to lease n . Pad unused decision opportunities with an empty segment and $\alpha_n = 0$, or equivalently restrict each conditional statement to histories where a lease is issued. Couple its executed path with the counterfactual path that follows the same candidate policy through its full default closure. Up to the next accepted renewal the paths have the same kernels, controls and observations; the renewal is a stopping time. Consequently, first harm in the realised segment is a subset of first harm under the counterfactual complete candidate continuation. Lemma 2 yields

$$\Pr(E_n \mid \mathcal{I}_{T_n}, D) \leq \alpha_n.$$

The segment events are disjoint. The tower property and the pathwise ledger give

$$\begin{aligned} \Pr\left(\bigcup_n E_n \mid D\right) &= \sum_n \mathbb{E}[\Pr(E_n \mid \mathcal{I}_{T_n}, D) \mid D] \\ &\leq \mathbb{E}\left[\sum_n \alpha_n \mid D\right] \leq \varepsilon. \end{aligned}$$

A bounded number of decision opportunities suffices; a countable version needs nonexplosion and monotone convergence. Independence of calibration from exogenous deployment randomness leaves the declared true kernels valid after fixing D , including for a policy chosen using that dataset. Define the calibration coverage event $E_{cov} = \{\theta_* \in \Theta(D)\}$. Averaging over datasets gives at most $\varepsilon \Pr(E_{cov}) + \Pr(E_{cov}^c) \leq \varepsilon + \delta$. This does not condition a local risk bound on a future deployment-dependent “good event.” $\square$

Meaning. Conditional on model coverage, a one-percent episode allowance gives an upper bound of one percent through many renewals. The price of this simple proof is double-counted prospective fallback risk. Section 12 quantifies this conservatism in a separate early-renewal audit. A more efficient refund, contingent budget or risk-to-go scheme would require its own composition proof and comparison with existing risk-budget literature.

Early revocation. An authority revocation before planned expiry is an admitted stop trigger, not automatically a lower-risk event. The policy and model must cover every allowed trigger and its response delay. For a simple nonnegative constant hazard, stopping earlier reduces exposure; for a cyber-physical process, stopping can create another hazard. The latter needs a separately specified event/model and local protective controller.

Renewal preserves the episode account and open exposure

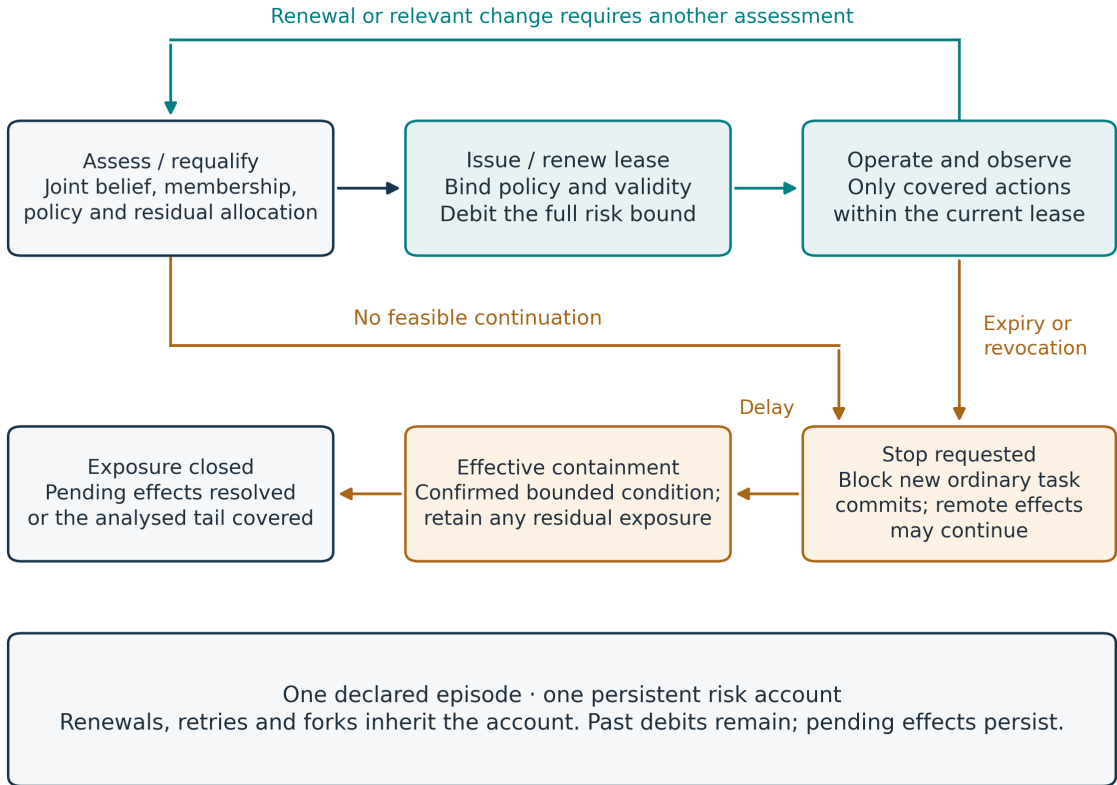
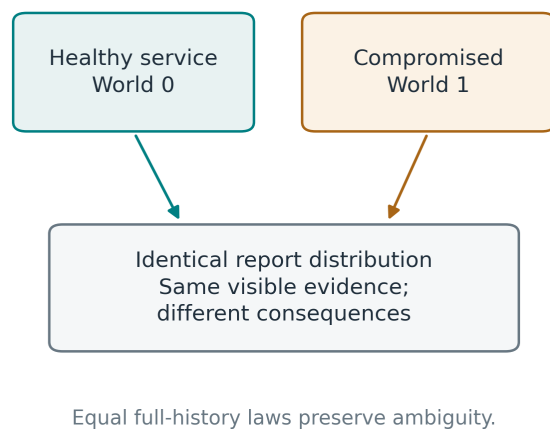


Figure 5: Assurance lifecycle with permanent episode debits. Renewal charges the new complete continuation. Stop requested, stop effective and exposure closed are distinct states; retirement does not erase pending effects.

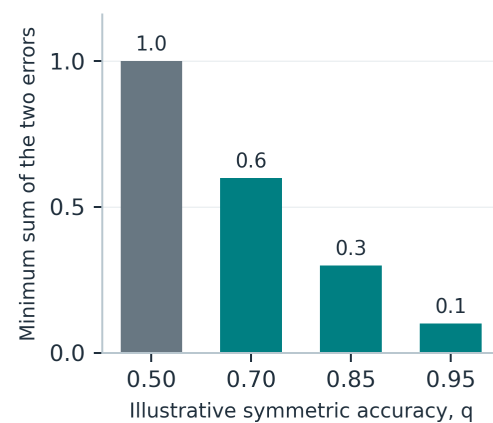
Information limits constrain useful admission

The monitor can distinguish worlds only through differences in their observable histories.

A An uninformative channel



B A discriminating probe



False admission in World 1 + false restriction in World 0 $\geq 1 - TV$
A probe needs authority, its own assurance, and justified observation likelihoods.

Figure 6: Observation limits and the value of an informative probe. Identical reports cannot distinguish healthy and compromised states. The numerical panel applies the total variation (TV) identity to a symmetric binary probe; it is not an empirical sensor evaluation.

9 Eligibility and controller refinement

Let $\mathcal{A}_t^{\text{mandate}}$ contain exact proposals satisfying the current principal, delegation, scope, epoch, expiry, configuration and resource requirements. Let

$$\mathcal{A}_t^{\text{assurance}} = \{a : \exists(h, \pi), K_t(a, h, \pi) = 1, \bar{r}_{t,j}(a, h, \pi) \leq \alpha_{t,j} \leq B_{t,j} \quad \forall j\}. \quad (9)$$

The predicate K_t checks applicability of the evidence/model bank, exact action and continuation bindings, containment availability, required observation channels, and a sound returned numerical bound. The qualifying (h, π) is retained in the lease. Eligible effects satisfy

$$\boxed{\mathcal{A}_t^{\text{eligible}} = \mathcal{A}_t^{\text{mandate}} \cap \mathcal{A}_t^{\text{assurance}}.} \quad (10)$$

The risk threshold acts on assurance; it cannot repair missing authority. Conversely, a mandate cannot supply missing evidence. Authority and assurance are evaluated against coherent state immediately before commit. Configuration, authority epoch, assurance generation, observation/model-bank commitment and episode account are distinct fields.

For several principals, each required permission must apply to the same bound continuation. Intersecting sets of initial actions after separately choosing incompatible local continuations is insufficient. Section 14 makes the common record explicit and separates mandate compatibility from the joint risk calculation.

Equations (9)–(10) use action-set shorthand. The implemented candidate also retains its selected duration and continuation, and both mandate and assurance are tested on that same complete candidate before projecting to the initial action. Section 14.2 gives the explicit common-record form. Separate existential choices of a mandate-compatible continuation and an assurance-compatible continuation would leave a quantifier gap.

Proposition 3 (controller refinement). Suppose the extended initial state projects to a state satisfying Paper 1’s invariant I , every old transition preserves I , and the new implementation: (i) changes each old action-admitting transition only by adding assurance predicates; (ii) performs new assurance-only transitions without changing old invariant state; and (iii) implements lifecycle/containment transitions through old preserving transitions or checks them separately. Then I remains an invariant of the extended controller. If every effect-producing transition also checks that the effect and its execution position are covered by the current bound continuation, with action-identifier replay protection, covered effects satisfy the implemented mandate–assurance conjunction.

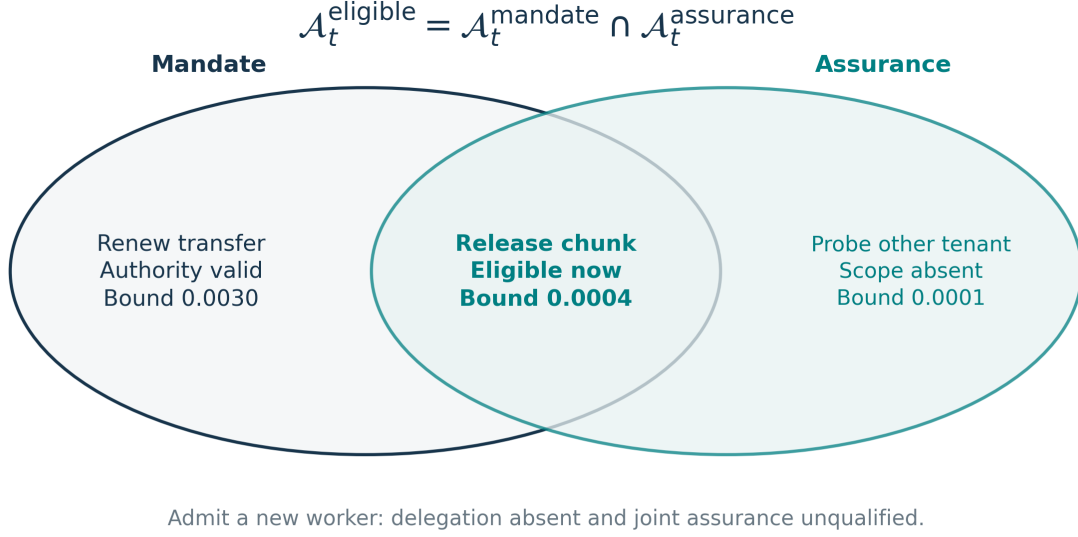
Proof. The initial projection satisfies I by assumption. A transition restricted by an extra guard is a subset of an old preserving relation. Assurance-only steps preserve the old state. The remaining transitions preserve I by the explicit third condition. Induction over transition sequences proves the claim. $\square$

This is an execution conformance result. It does not make the observations correct, establish model containment, or extend a local atomic transaction to an unacknowledged remote API. Release adapters need explicit pending, idempotency and unresolved-outcome semantics; those states enter Q .

Controller continuity. Paper 1’s deterministic controller separates a bounded decision kernel from protected stateful execution [1]. Here the assurance supervisor supplies a conditional continuation value and the episode account; determinism for fixed explicit inputs does not make the hidden

dynamics deterministic. Paper 3 separates pure eligibility evaluation, protected admission and resource-side effect adapters [14]. Proposition 3 preserves the projected authority and resource invariants. Transferring Theorem 2 additionally requires preservation of the observation and proposal-selection laws, covered transition kernels, timing and pending exposure. Permitted-trace inclusion alone does not preserve a numerical probability bound.

Eligibility requires both authority and assurance



Illustrative threshold test

Upper bound ≤ 0.001 for the specified disclosure event and full exposure.
Each value assumes an applicable model and enforced continuation policy.
The bound cannot supply authority; the controller rechecks all conditions at commit.

Figure 7: Eligible actions satisfy both mandate and assurance conditions. The examples distinguish permission from evidence-supported risk. The intersection does not imply statistical independence of the two decisions.

10 Finite-bank algorithm and computational cost

1. Declare the episode, harm(s), owner, initial state, model bank, calibration procedure, lifecycle menu, exact action/policy library and closing condition. Set $B = \varepsilon$.
2. Retain the jointly calibrated complete models. If the bank is empty, supply no new certificate; execute the predeclared protective response.
3. For each actual observation, missingness event and executed control, update every retained full posterior, including H . Apply covered joint lifecycle kernels; preserve unchanged evidence scope and pending work.
4. On a proposal, test current authority and exact action identity. For each candidate continuation that passes the authority and applicability checks, calculate its full global incremental first-harm bound, including containment and queued exposure.

5. Choose a candidate that meets all hard requirements and whose harm-specific debits fit the remaining accounts. Selection can maximise a declared expected useful-work criterion; a proposal’s verbal confidence is irrelevant to the bound.
6. Atomically debit the episode account and issue the action/policy-bound lease. Recheck authority, configuration, assurance generation, reservation, identifier and expiry at the effect commit.
7. Follow the admitted policy until an accepted replacement. If renewal fails, observations are late, or the solver times out, follow the currently covered stop continuation. Retain unresolved exposure.
8. Close only at the declared terminal condition or with an explicitly charged residual-tail argument. Retain all debits throughout the episode. Export distinct observation, decision, command and confirmed-effect records.

For a dense state space of size S and M retained models, a one-step filter costs $O(MS^2)$. Replaying t historical steps costs $O(tMS^2)$ before comparable reachability work. Retaining a sufficient joint posterior avoids this repeated past computation. It does not remove the cost of recalculating a changed future kernel. Worker combinations and observation-dependent policy trees can grow exponentially. The practical baseline therefore bounds membership, queue size, horizon and the policy menu.

An assurance solver receives a finite deadline. The old covered continuation remains in force during computation. A timeout cannot silently extend operation. The policy witness is part of the lease: existence of one qualifying continuation does not authorise substituting a different continuation after issue.

11 Counterexamples and information limits

11.1 Omitting containment delay

With constant hazard rate λ , planned operation h and continued exposure during delay d , risk is $1 - e^{-\lambda(h+d)}$. A planner that admits $h = -\log(1 - \varepsilon)/\lambda$ ignoring delay actually incurs $1 - (1 - \varepsilon)e^{-\lambda d} > \varepsilon$. For $\lambda = 0.001$, $\varepsilon = 0.01$, $d = 2$, the valid pre-stop period is about 8.0503 minutes, not 10.0503. A negative feasible period means even immediate containment misses the chosen allowance under those assumptions.

11.2 A fresh account at renewal

Ten independent disjoint segments each given a fresh one-percent allowance yield episode first-harm probability $1 - 0.99^{10} \approx 0.09562$. Independence is used only for this illustrative equality. The persistent-account bound does not require it.

11.3 Duplicated common evidence

Let one provider be compromised with probability p ; all its workers inherit that condition. For a harm requiring two compromised roles in a joint release, the true joint compromise probability is p . Replacing the shared state by independent copies gives p^2 . At $p = 0.1$, this changes 0.1 to 0.01. For a different harm defined as “either role compromised,” independence can overestimate instead. Dependence must be modelled with respect to the actual harm event, rather than asserted to increase risk universally.

11.4 Discarding a retired worker's queue

A retired worker has one already accepted release that will execute with probability $q > 0$ and violate isolation conditional on execution with probability $p > 0$. Deleting its state and queue reports zero remaining exposure although the remaining harm probability is pq . Withdrawal of future permission does not cancel an accepted remote effect.

11.5 Conditioning on a silent alarm

Suppose first harm has occurred with probability 0.2 but the alarm never reports it. The posterior is still 0.2 on $H = 1$. Setting it to zero because no alarm arrived asserts knowledge the system does not possess. A posterior filter may represent this uncertainty; an audit receipt cannot correct it by signature alone.

11.6 Observation indistinguishability

If two models yield identical observation laws, release has the same admission probability q under both. When release causes harm with conditional probability p in one model, uniform risk requires $pq \leq \varepsilon$. This applies the observation limitation in [12].

The equal-admission conclusion requires the same decision rule and the same distribution of its complete decision-relevant transcript under both models, including any probing interaction and admissible randomisation. Identical monitor reports alone are insufficient if another input reveals the model. This is the statistical decision-interface counterpart of Paper 1's private-interface limitation [1]; privacy of a proof is a separate property.

11.7 Candidate-specific error reset

For ten independent candidate tests, each with erroneous-admission probability 0.05, the probability of at least one erroneous admission is $1 - 0.95^{10} \approx 0.4013$. It is also the selected-error probability when the selector can exploit any erroneous admission. This illustrates the selection-sensitive distinction in [12]. Our shared bank does not spend a fresh calibration allowance on each exact query; misspecification remains outside coverage.

11.8 Undisclosed proposer information

Suppose a fresh hidden dangerous condition occurs independently with probability 0.01 at each of 100 ticks. A proposer privately observes it and requests release only at the first dangerous tick. A supervisor that ignores this selection may charge 0.01 once. The true probability of a harmful admitted release is $1 - 0.99^{100}$, about 0.634. The proposal itself is informative. Complete mediation alone does not fix a filter that treats such a request as independent of hidden state. This is why the theorem either needs the declared shared history restriction or an explicit selection likelihood.

11.9 What additional monitoring can buy

For admission event A , let $d_{TV}(P, Q)$ denote total variation distance between observation laws P and Q . Then $|P(A) - Q(A)| \leq d_{TV}(P, Q)$ implies $\alpha + \beta \geq 1 - d_{TV}(P, Q)$, with $\alpha = Q(A)$ and $\beta = 1 - P(A)$ [12].

Here P is the complete observation law under the hypothesis where admission is appropriate, and Q is the law where it is inappropriate, for one fixed probing and admission mechanism. Thus α is false admission under Q and β is false refusal under P . These testing errors concern distinguishability; they become an episode harm claim only through a separate consequence and exposure model.

For one symmetric binary probe with correctness $q \geq 1/2$, the distance is $2q - 1$. Correctness values 0.50, 0.70, 0.85 and 0.95 yield lower bounds 1.00, 0.60, 0.30 and 0.10 on the error sum. These are exact teaching calculations. The probe's cost, delay and possible effects remain separate obligations. The experiments compare monitoring schedules; they do not solve a joint optimal probe-design problem.

12 Reproducible synthetic evaluation

12.1 Model and event order

The main sweep contains 432 scenarios: six actual models, two worker schedules, three containment delays, two missingness rates and six policies. Each scenario has six decision epochs. Across the sweep, numerical enumeration visits 120,060 terminal observation histories. These are paths across repeated scenarios, not independent empirical trials or unique controller states.

The hidden state is (Z, H) , with provider condition Z either good or degraded and H the absorbing, unobserved first-harm flag. No policy observes H . Initial degraded probability is 0.08. Between epochs, good-to-degraded transition probability is 0.02 and degraded-to-good probability is 0.10. The provider condition remains constant within a segment. This is a finite decision model with continuous exposure inside each segment, not a continuous-time model in which provider condition changes at arbitrary instants.

The complete model bank is the product of symmetric sensor accuracies $\{0.70, 0.85, 0.95\}$ and degraded hazard rates $\{0.04, 0.08\}$ per worker-time unit. Good-state hazard is 0.0005. Missingness is either zero or 0.35 and is independent of condition, harm and action. Present reports identify provider condition correctly with the model's stated accuracy. These are synthetic parameters without asserted production meaning.

At the first epoch the supervisor uses the initial prior; between later epochs it predicts provider state. At each epoch it receives a report or declared absence, updates all retained joint posteriors, reads the predeclared membership schedule, allocates remaining risk over remaining epochs, selects a duration and debits its full continuation. The segment then executes through containment or is refused. Segment closure eliminates its modelled exposure; the declared six-epoch task and its permanent account continue until the final episode closure. The static schedule has one worker throughout. The changing schedule has 1, 1, 2, 3, 2 and 2 workers. Its membership changes occur after closure of the preceding segment. Pending work during early renewal is tested separately in Section 12.4.

For n active workers, duration h and delay d , conditional harm probability is $1 - \exp[-\lambda_Z n(h + d)]$. Risk from belief b is

$$\rho_\theta(b, n, h, d) = \sum_{z \in \{G, D\}} b(z, 0) \{1 - \exp[-\lambda_z n(h + d)]\}.$$

The supervisor maximises this value over the retained bank. It does not condition its belief on unobserved survival. Worker exposures share one latent provider variable; the conditional

additive rate is an explicit modelling choice. A copied provider report is never counted as an independent measurement.

The episode allowance is 0.05. At epoch $t \in \{0, \dots, 5\}$, the local allocation is remaining allowance divided by $6 - t$. The policy chooses the longest admissible duration from $\{1, 2, 4\}$, unless its menu is fixed at one. This heuristic preserves the accounting contract but is not claimed to maximise episode utility. Containment delays are zero, one or two abstract time units.

12.2 Matched policies and metrics

Table 2: Policies in the synthetic sweep. The code identifier `full_requalification` denotes full-history recomputation only. It does not perform new physical qualification or reproduce a published certification algorithm.

Policy	Observation and decision rule	Interpretation
Fixed expiry	Full bank, report every epoch, duration one	Short-duration reference with the same ledger
Periodic monitor	Full bank, reports only at epochs 0, 2 and 4; adaptive duration	Reduced observation schedule
Nominal point	Accuracy 0.85 and degraded rate 0.04 assumed; report every epoch	Tests unsupported model certainty
Robust reuse	Full bank, incremental joint filtering, adaptive duration	Reference implementation of the restricted construction
Full-history recomputation	Same full bank and policy; replay all actual history for each decision	Controlled computational comparator
State oracle	Actual provider condition and true model known; H still hidden	Diagnostic information advantage

Admitted work is expected active workers multiplied by permitted operating duration, summed over epochs. It includes work on histories where harm occurs. Harm-free completed work additionally weights each segment by absence of first harm through its closure. Both are abstract work measures, not observed application value. The released data also record refusal frequency, admitted duration, ledger charges, belief operations and runtime. Episodes with complete refusal remain in every metric.

Probe cost is not deducted from work. The oracle receives extra information without a sensing charge. Thus the comparisons are not cost-matched net-utility optimisation. The robust calculation is a specialised finite-bank shield. The experiments do not reproduce RAO*, interval-POMDP shielding or a full decentralised planner.

12.3 Main results and the value of reuse

All 360 scenarios using supported models remain below 0.05. This set comprises every policy except the nominal-point baseline and includes the diagnostic oracle. Its largest computed harm probability is 0.037257. The robust-reuse policy and the full-history comparator reach at most 0.034535. The nominal baseline exceeds 0.05 in 15 of its 72 scenarios, reaching 0.065973.

Table 3: Equal-weight descriptive means over 72 synthetic settings per policy. They do not estimate population averages. Harm probabilities are obtained by model enumeration with floating-point arithmetic.

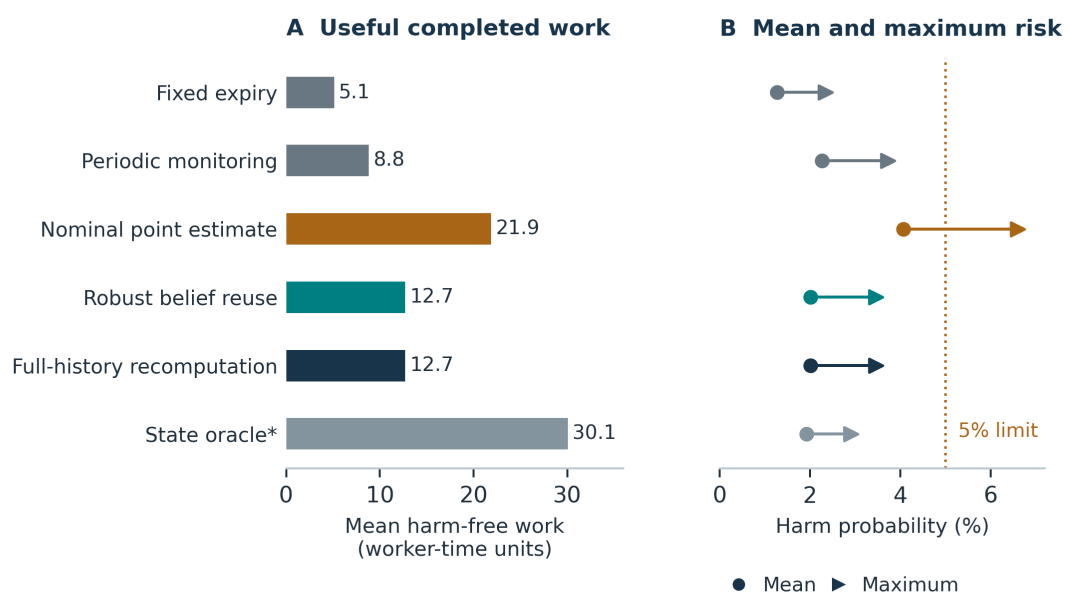
Policy	Mean admitted worker-time	Mean harm probability	Maximum harm probability
Fixed expiry	5.158	0.012719	0.023445
Periodic monitor	8.930	0.022669	0.037257
Nominal point	22.311	0.040720	0.065973
Robust reuse	12.845	0.020052	0.034535
Full-history recomputation	12.845	0.020052	0.034535
State oracle	30.457	0.019205	0.029039

All 72 reuse/recomputation pairs have identical decision digests, work and risk values, with zero recorded posterior difference. One belief operation is a state-vector transformation for one model: prediction, observation update or propagation through an executed segment. Counts are weighted by observation-history probability. Both methods include propagation through each newly admitted segment; full replay additionally repeats past transformations. Truth-state propagation, risk evaluation and validation-only duplicate calculations are excluded. Across the 72 settings, incremental filtering averages 88.27 operations per episode, versus 289.62 for full-history replay. Paired reductions range from 68.03 to 70.30 percent, with a mean of 69.46 percent. This verifies sufficient-statistic reuse in the tested construction. It does not establish a new filtering method or a speed advantage over an efficient existing shield. Reachability must still be recalculated when future dependencies change.

A representative changing-membership case uses delay one, missingness 0.35, actual accuracy 0.85 and degraded hazard 0.08. Robust reuse admits 10.952 worker-time with harm probability 0.022026 and 2.788 expected refused epochs. Fixed expiry admits 5.629 with harm probability 0.016841. Periodic monitoring admits 7.043 with harm probability 0.026088. The nominal policy admits 24.802 but reaches harm probability 0.051477. These differences show why reporting work together with risk and refusals matters. The result does not establish that additional monitoring universally reduces unconditional harm: it also changes admitted exposure.

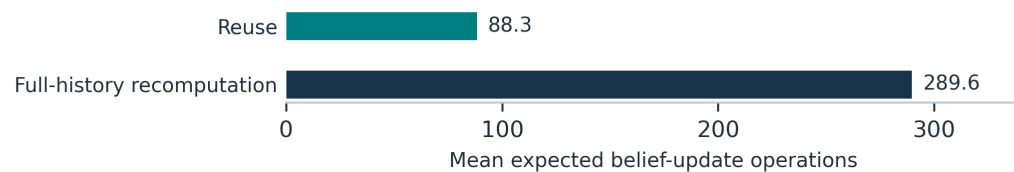
Matched synthetic cases expose the utility-risk trade-off

Finite observation-tree enumeration in floating point; 72 declared settings per policy.



C Reuse and full-history recomputation make the same decisions

All 72 paired settings: identical decision digests, harm probabilities and completed work.



*The state oracle is a diagnostic comparator with privileged information.

Figure 8: Synthetic policy comparison. Each policy has 72 declared settings. Mean harm-free completed work, mean risk and maximum risk are descriptive summaries; the threshold applies to each episode. The computational panel compares incremental reuse with complete history replay under identical decisions.

12.4 Early renewal and inherited pending exposure

A separate 18-case audit tests overlapping forecast horizons. Two workers initially receive a lease with operation two and delay one. Renewal occurs at time 0.5, 1 or 1.5. One original worker retires while leaving a queue exposed for one further time unit. New active membership is one, two or three workers; new operation is one or two units. There is no new observation and provider condition does not change at renewal. The full joint posterior, including hidden harm mass, is retained.

The old full forecast covers six worker-time and remains permanently charged. The new forecast includes active work through containment plus the inherited queue. Actual risk comprises harm in the old executed prefix plus the new incremental exposure. The sum of old and new debits bounds actual risk in all 18 cases. These cases use a separate 0.10 allowance: maximum actual risk is 0.057684 and maximum total debit is 0.078506. They must not be judged against the main sweep's 0.05 threshold.

For renewal at time one, one new active worker and new duration two, the retained original debit is 0.033253. The new debit is 0.020505, giving total charge 0.053758. Maximum actual episode risk over the bank is 0.033253. Erasing the inherited queue would report a new debit of 0.015923, an underestimate of 0.004582. The old conservative debit means this particular error need not breach the total episode threshold. It still makes the new local claim incorrect. The gap between total charge and actual risk also exposes the cost of the no-refund construction.

This audit supports the state-transfer and charging calculation. It is not a network implementation with uncertain acknowledgements or arbitrary concurrent policy replacement.

12.5 Calibration and deliberate model failure

The main sweep uses the whole containing bank, so its coverage error is zero conditional on bank correctness. A separate illustration uses independently labelled synthetic calibration counts: 86 correct reports from 100 good states and 87 from 100 degraded states. Two exact binomial confidence intervals each use error allowance 0.025. Their simultaneous error is at most 0.05 by a union bound. The resulting intervals are $[0.763848, 0.927993]$ and $[0.775728, 0.935269]$. They retain accuracy 0.85 and both hazard values. These data calibrate the observation law, not the hazard rates.

Enumeration of the independent binomial sampling laws gives joint coverage 0.957686, 0.952475 and 0.965517 at accuracies 0.70, 0.85 and 0.95. Each exceeds 0.95. In the representative deployment case, the retained bank admits 20.366 worker-time with harm probability 0.034348 when truth has accuracy 0.85 and degraded rate 0.08. This conditional example is not an unconditional performance estimate. Applying Theorem 2 with a 0.05 calibration error and a 0.05 operational allocation yields an upper bound of 0.10, not 0.05. A deployment seeking a total 0.05 bound must divide that total appropriately.

Finally, hold the supervisor's original bank fixed and deliberately move truth outside it. Actual accuracy/degraded-rate pairs (0.50, 0.08), (0.15, 0.08), (0.50, 0.20) and (0.15, 0.20) give harm probabilities 0.044629, 0.070828, 0.068157 and 0.098786 in the representative setting. Three exceed 0.05. The first happens to remain below it but has no guarantee. Arbitrary sensor corruption or a changed hazard model cannot be hidden inside a statistical coverage allowance.

12.6 Numerical and empirical limits

All observation histories are enumerated; there is no Monte Carlo sampling or empirical rare-event estimate. Arithmetic is double precision. Maximum terminal branch-mass error is below 2×10^{-15} , and threshold comparisons allow 10^{-13} slack. These checks do not provide outward-rounded numerical certification. A deployed certificate needs explicit error enclosure or a verified numerical method. The analytical theorem assumes sound bounds, while these computations provide finite numerical evidence.

The suite has one shared provider variable, bounded membership, six decision epochs, predictable schedules and one harm. It does not evaluate live AI agents, Byzantine reports, continuous latent changes within a lease, observation-dependent missingness, unbounded spawning or real containment hardware. The complete recorded run took about 15 seconds; that is not an industrial overhead benchmark. The package supplies parameters, code, raw outcomes, decision hashes and environment metadata so these results can be reproduced and extended.

13 Finite execution and accounting checks

13.1 Controller abstraction

The controller model has two worker identities and two immutable action identifiers. Worker zero begins admitted; worker one can join and retire once. Both share a money account and one permanent episode risk account. Authority can be revoked and reissued in a new epoch. Membership and evidence changes advance a separate assurance generation. One bounded configuration/policy version change is included.

One current joint-system lease binds its serial, authority epoch, assurance generation, configuration and exclusive expiry. Each issue or renewal consumes one abstract risk token. A token has no assigned probability. The issuer is an ideal oracle assumed to check the complete continuation, including unresolved exposure. Ordinary action reservation consumes monetary capacity, not another risk token. Cancellation can release uncommitted money but cannot refund a lease debit.

Dispatch rechecks the current record and creates a pending remote outcome. A separate acknowledgement settles it. Stop requested and stop effective are separate states. Effective stop blocks further gateway exposure but neither reverses accepted effects nor resolves their outcomes. Retirement preserves pending records. Recovery after effective stop requires resolved outcomes, current authority and a new lease. The finite model does not assign a duration to containment; the stochastic model handles that obligation separately.

13.2 Exhaustive results

Breadth-first exploration stores each reachable state once and checks every generated transition. Rejected state-preserving self-loops are omitted. Each row in Table 4 is a separate complete exploration.

Table 4: Finite controller checks. All stated baseline invariants hold in each exploration. Counts must not be added as disjoint parts of one state space.

Money budget	Episode risk tokens	Reachable states	Transitions checked
1	1	7,300	25,501
2	1	10,134	37,121
1	2	16,076	58,214
2	2	24,448	94,758

The invariants cover aggregate money, permanent lease debits, current authority and lease context, exact payload, single dispatch per identifier, no dispatch after a stop request, preserved pending outcomes, and agreement between history and accounting. Nine direct boundary checks include dispatch before expiry, refusal exactly at expiry, post-reservation context changes and pending-outcome retention after effective stop.

Twelve separate control omissions each yield a shortest counterexample of two to five transitions. Omitting epoch, assurance generation, version, expiry or current-lease checks allows stale dispatch. Omitting monetary or risk guards exceeds the corresponding allocation. Resetting the risk ledger on membership change breaks permanent accounting. Removing replay protection repeats an action. Removing payload binding substitutes its recipient. Removing the stop gate permits dispatch after a stop request. Deleting pending state at effective stop loses an unresolved outcome.

For example, the stale-generation trace issues a lease, admits worker one, reserves an action under the old lease and dispatches it without checking the changed generation. The pending-loss trace issues, reserves and dispatches, then revokes authority and makes stop effective while incorrectly deleting the pending record. Full states and shortest traces are included in the package.

These checks validate a finite enforcement abstraction. They do not validate the oracle’s risk estimate, observation accuracy, cryptography or distributed consistency. The chosen single-current-lease rule is one protocol design; overlapping leases require a separate protocol. The finite counts do not prove an unbounded implementation. Proposition 3 supplies the separate inductive refinement condition, subject to its stated assumptions.

14 Assurance across organisational boundaries

14.1 The unit of assurance is the joint episode

The relevant safety boundary follows the effects and dependencies of a task. It need not coincide with a company’s boundary. Agents within one organisation can use different observations and share an external provider. Agents from several organisations can operate within one explicitly governed episode. The number of legal principals alone therefore neither establishes nor defeats the assurance argument.

Theorem 2 applies when the participating agents and their environment satisfy one declared joint model family, information contract, continuation policy and episode account. This is a **logical supervision boundary**. Its components may be physically distributed, provided their implementation refines those semantics. A federation of independent local supervisors does not acquire the theorem merely by exchanging signed approvals. The experiments in Sections 12–13 exercise the logical construction; they do not implement a cross-organisation protocol.

For one organisation, an accountable principal can establish the task, harm limits and delegation scope. A protected supervisor evaluates the joint worker pool, and gateways mediate its effects. Workers share the episode ledger and retain their separate identities and permissions. Adding a worker changes the joint state through Section 6; it does not create a fresh allowance. A single organisation still needs shared-dependency modelling and a declared information scope. Corporate ownership supplies neither statistical independence nor complete telemetry.

For two organisations, mandate formation remains separate. The data owner can permit a particular disclosure, while the recipient authorises processing and resource use within its own boundary. Each permission applies to an identified part of the same operation. Neither organisation can grant the other’s authority. Their policies must also admit a compatible continuation, including acknowledgements, retention, revocation and containment. Section 14.3 gives a concrete construction.

14.2 One joint action record and compatible continuation witnesses

For the shared logical supervision construction, let $g = (a, h, \pi, \nu)$ be a complete candidate record: exact initial action, duration, joint continuation and bound context. The context ν includes the episode, participants, dependencies, harm definitions, harm-specific debits, threshold policy and relevant versions. Let $\mathcal{U}_t$ be the declared candidate-record universe, and let $P(g)$ identify the principals whose permission is required. This mapping follows the protected resource and authority policy; an agent cannot remove a required principal by changing its proposal. Principal p evaluates $M_{p,t}(g)$ against its current mandate and the obligations assigned to it.

The cross-organisation form of the intersection is

$$\begin{aligned}\mathcal{G}_t^{\text{mandate}} &= \{g \in \mathcal{U}_t : M_{p,t}(g) = 1 \ \forall p \in P(g)\}, \\ \mathcal{G}_t^{\text{assurance}} &= \{g \in \mathcal{U}_t : K_t^{\text{joint}}(g) = 1, \\ &\quad \bar{r}_{t,j}^{\text{joint}}(g) \leq \alpha_{t,j} \leq B_{t,j} \ \forall j\}, \\ \mathcal{G}_t^{\text{eligible}} &= \mathcal{G}_t^{\text{mandate}} \cap \mathcal{G}_t^{\text{assurance}}, \\ \mathcal{A}_t^{\text{eligible}} &= \{a(g) : g \in \mathcal{G}_t^{\text{eligible}}\}.\end{aligned}\tag{11}$$

The gateway retains and enforces the chosen record g after projecting to the initial action. Equation (11) is an interface specialisation of (9)–(10), not an additional probability theorem. Every approving principal and the joint evaluator must refer to that same record. Separate statements that each party can find *some* acceptable continuation for a do not establish a jointly acceptable continuation. One party may rely on immediate deletion while another requires continued processing; individually acceptable witnesses can conflict. Independent implementations can nevertheless use different role-specific local continuations when a justified mapping and composition realise the same covered joint behaviour. Section 14.5 explains this contract-based interface; it does not require identical policies or disclosure of all private implementation state.

The joint assurance predicate covers the whole continuing exposure, including shared providers and remote queues. Local restrictions and their triggers must form part of the evaluated and enforced record g ; marginal local scores cannot replace the joint predicate. Unilateral refusal or stopping after certification can change a partner’s risk and must be covered by the continuation. If several harms matter, each has its own definition, authorised threshold and account. For example, confidentiality loss and a damaging physical shutdown require separate constraints. Agreement on a weighted average does not establish either one.

Budget ownership also needs an explicit rule. The parties can authorise a common task account with named responsibility, or maintain separately governed harm-specific accounts evaluated against the same joint continuation. Within each account, allocation and debit must be coherent before effects occur. Replicas, agents and organisations cannot each spend a copy of the same remaining allowance. Each declared harm constraint requires one coherent logical account; a particular ledger technology is not implied.

14.3 A bilateral data release example

Suppose engineering company A owns a protected design dataset and company B supplies a simulation service. A’s agent requests release of chunk 17 to B’s named job. A’s mandate limits dataset, purpose, recipient and delegation. B’s mandate permits the named workload, resource allocation and receiving service. A business identity credential can authenticate the represented organisation; a scoped delegation record connects that principal to the running instance. The gate still checks the precise operation and current policy state.

Before release, the parties bind the endpoint, payload commitment, job configuration, permitted onward flows, operation deadline and containment obligations to one record. Accepted evidence covers the receiving environment and the observable queue/stop interface. The joint estimator retains dependence on a shared hosting provider. A valid identity or a signed compliance statement alone does not determine the receiving environment’s current condition.

If the full continuation meets every declared constraint, the account debits its bound before dispatch. A’s gateway checks current authority and the selected record. B’s gateway checks that the incoming request matches the same record and B’s local permission. Their acknowledgements distinguish accepted, executing, stopped and closed states. The joint model includes the intermediate states and their exposure. A common commit identifier assists correlation and replay control; it does not make the remote exchange physically atomic.

If A revokes while a chunk is queued, A prevents further release and invokes the previously covered response. B’s actual reaction delay and any work already accepted remain in Q . An acknowledgement lost in transit leaves the outcome unresolved. Exposure closes only at the declared condition, or with the charged residual tail. If B can bypass the gateway or conceal onward disclosure outside the covered model, the quantitative claim does not apply to that behaviour. Hardware isolation, additional observation or a narrower permitted workflow may be needed before admission is supportable.

Each side can verify signatures, delegation, policy versions, record equality and replay status. These checks establish provenance and conformance of the submitted records. Establishing the risk claim also requires validation of the model, observation process, numerical bound and execution interface. Privacy-preserving evidence may conceal inputs, but it must preserve the information needed by the declared assurance calculation. A private proof cannot supply a missing physical observation.

14.4 Architecture patterns and bounded swarms

Three patterns clarify the implementation choices. **Shared supervision** places one logical joint estimator, continuation evaluator and episode account behind local gateways. This most directly matches Theorem 2. The service can be operated by an agreed party, provided governance, confidentiality and availability requirements are met. **Federated execution with joint supervision** keeps authority and effect mediation within each organisation while exchanging the evidence and state needed for the same logical decision. Delayed messages, authority changes

and duplicate requests must be represented in the model and protocol. Both patterns need a demonstrated refinement from distributed events to the model’s decision order. During renewal, mixed old and new policies must either be excluded by the protocol or covered as an explicit coordination phase. Debiting a new lease alone does not make its installation atomic.

Contract-based federation of independent multi-agent systems (MAS) keeps each organisation’s planning, mandate evaluation, assurance and gateways under its own control. Parties agree on a transaction interface and exchange role-specific eligibility assertions, as developed in Section 14.5. Compositional local assurance is a possible verification method for this pattern, rather than a further deployment topology. Verified component contracts would state assumptions about peers and guarantees about local effects. Mutual assumptions must be established initially and preserved together; circular promises do not discharge them. Cross-boundary hazards, shared causes, message faults and recovery timing require a composition argument. Existing local risk values are insufficient when obtained under incompatible policies or observations. This pattern is a proposed extension unless it is shown to refine the covered joint construction.

Figure 9 illustrates the shared-supervision cases by organisational scope; the independent-MAS pattern uses the additional contract interface described below. A bounded swarm is a changing population of agents that delegates and coordinates within a declared task. Consider a factory operator, a logistics provider and an equipment service company scheduling mobile robots through a shared aisle. Each organisation can authorise its own robot. Two locally permitted moves can still cause a collision, block an emergency route or exceed shared capacity. The joint action record must cover simultaneous reservations, relevant observations and the continuation after communication loss. This physical example illustrates the architecture; the present data-release experiment does not validate robot safety.

A task coordinator may propose assignments, but protected gateways enforce the agreed task contract. A covered join adds the agent’s identity, authority, dependencies and state through the joint lifecycle kernel. A departing agent loses new permissions while its pending operations remain represented. A fork inherits the episode reference and consumes available allocation through the same account. The supported population, communication behaviour and policy library remain bounded. A new organisation, provider or interaction outside that family requires a new coverage argument before its effects enter the certified workflow. Pairwise compatibility does not establish safety of higher-order interactions. Reducing a large swarm to counts or aggregate states requires a conservative abstraction argument; the current experiments establish no such scaling result.

Common infrastructure can couple agents across companies. Two agents using the same model service, sensor or cloud do not generate independent evidence merely because different principals sign their reports. Likewise, a partner’s proposal can reveal private telemetry. Its selection mechanism must be included in the observation model, or all information used to select the action must meet Section 4’s declared-history restriction. Merely signing a proposal does not resolve this issue. Coarse certified bounds could support another sound interface, but their coverage and composition require proof.

On loss of communication, a gateway permits only the behaviour already covered by its current record and initiates the specified local response when required. Permitted deadlines must account for clock and message uncertainty. A quorum or replicated database may protect accounting under its stated failure model; it cannot establish that a remote actuator stopped. Global atomicity, bounded remote stopping and protection against colluding or Byzantine participants require explicit assumptions and validation. An unrestricted, open-ended swarm is outside the finite construction.

From one principal to bounded cross-organisation systems

Logical architecture proposal; distributed enforcement requires a separate protocol.

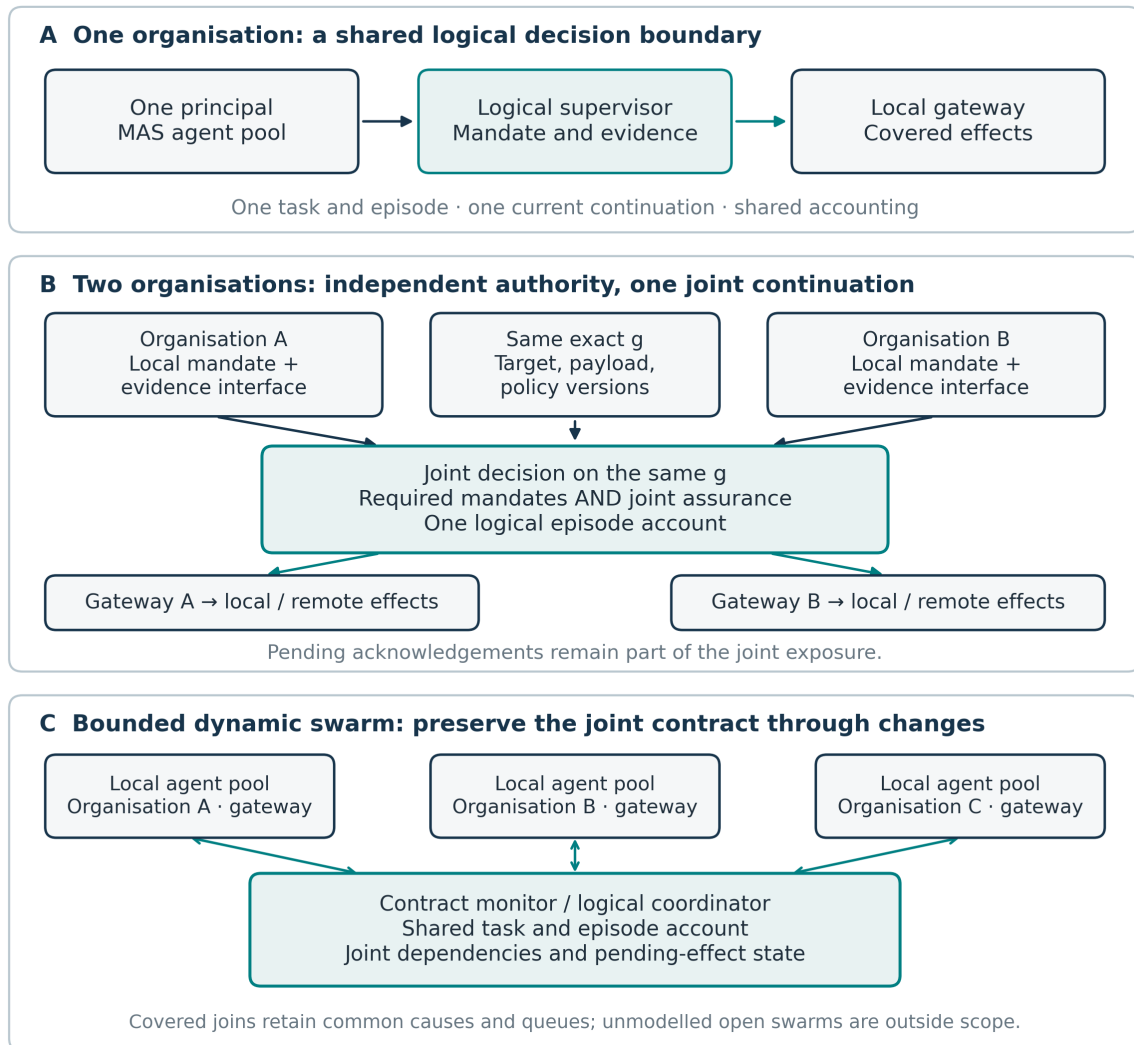


Figure 9: Proposed supervision patterns for multi-agent systems within one organisation, a bilateral workflow and a bounded multi-organisation swarm. All permissions refer to one exact continuation. Joint assurance retains shared dependencies and unresolved effects; coherent episode accounting prevents allowance duplication. Local gateways retain authority over their own effects. The figure specifies a logical architecture, not a validated distributed deployment.

14.5 Contracts and reciprocal assertions between independent systems

Contract-based federation allows A and B to retain different planners, worker pools, wallets, policy engines and assurance implementations. They exchange bounded claims about a transaction instead of exposing all internal state. Four objects must remain distinct. A **business agreement** identifies parties, obligations and governance. A **mandate** grants scoped authority to represent a principal. An **assurance contract** states model assumptions, named harms, risk limits and covered responses. A **transaction protocol contract** specifies permitted messages, effects, state transitions and timing. The first can require the other three; it does not implement them. Here a machine-readable contract is a versioned specification, without implying a blockchain or a particular legal effect.

For a transaction, the parties agree on an interface contract $\mathcal{C}$: identities and required principals, operation semantics, payload and destination, allowed sequences, deadlines, revocation behaviour, failure responses, evidence requirements and closing conditions. Their local continuations π_A and π_B may differ. A's release step and B's receive-and-process step are also different actions. Their local eligible sets therefore need not share an action universe and cannot simply be intersected. Each local action must pass its own mandate–assurance intersection, while the composed transaction needs a separate compatibility and assurance decision.

A refinement map relates each local continuation to its externally relevant obligations in $\mathcal{C}$. The maps must cover the actual composition, including queues, interleavings, private-information effects and shared dependencies. Section 14.2's requirement for one witness means a compatible **joint behaviour**, not identical internal software or identical local policies. Theorem 2 applies only if the implementation realises a covered joint continuation and preserves its accounting. An interface contract with compatible message types alone does not establish that condition. A fully decentralised assume–guarantee proof remains an extension.

The proposed protocol uses a **local eligibility assertion** from each required domain. It states that the identified local action and continuation passed both local mandate and assurance checks for the specified context. It binds the organisation, authorised assertion service and its verification key, agent instance and instance key; transaction and episode identifiers; contract version; durable local operation identifier; payload, target, peer and audience challenge; local continuation commitment; and mandate epoch and policy version. A transaction challenge prevents reuse across exchanges. Protected consumption state prevents replay of the same operation. To enforce a one-time logical effect, the identifier must designate a contract-defined operation slot whose identity cannot be reset by a caller, retry or replacement instance.

The assurance claim binds the harm definition, exposure horizon through containment and pending effects, numerical bound, calibration/coverage allowance, evidence/model commitments, assurance generation and validity interval. Its accounting reference distinguishes readiness or reservation from an already permanent lease debit. Required stop and acknowledgement behaviour form part of the claim. A readiness assertion, admission record, execution receipt and closure record have different meanings and must have distinct types. A generic claim that an organisation's MAS is safe provides none of these bindings.

The receiving domain checks who may issue that assertion, the protected record's integrity, its intended audience and transaction challenge, current status and its compatibility with local obligations. Its reliance policy states which claims may be self-asserted and which require independently assessed controls, execution attestation or a proof of an agreed computation. These forms of evidence support different propositions. An authenticated assertion attributes a claim; an attestation concerns a measured configuration; a computation proof concerns committed inputs and a specified computation. None alone establishes truthful sensing, model coverage, correct remote effects or the absence of bypass paths. Assertions may be encoded as verifiable

credentials or another authenticated format. A credential format does not itself define the assurance semantics.

In the bilateral example, B presents readiness for the exact receive-and-process obligation. A checks that evidence before considering release; B separately checks A's authority and transfer conditions. Protected services bind the two decisions to the transaction contract. Each gateway rechecks its current local context and the peer conditions required by $\mathcal{C}$ at its effect commit, using only the evidence age and message delay permitted by that contract. The protocol covers the period between preparation and effect, including one side proceeding while the other fails. If a lease has already been issued, its debit remains permanent under Section 8 even when the business transaction aborts. A receipt confirms only its stated event. A missing receipt leaves exposure unresolved; it does not create a new episode or refund risk.

These assertions make cross-boundary conformance inspectable while allowing internal implementations to differ. The remaining global obligation is substantive: each relied-upon local claim must stay valid under the actual interacting execution, and their composition must cover the named joint harm. Local risk bounds or contract signatures do not supply that argument by aggregation. Section 15.3 specialises the evidence interface for model evaluations and claims about deployed safety mechanisms, including closed services.

14.6 Identity delegation and verifiable evidence graphs

Digital organisational identity identifies the represented principal and connects it to evidence accepted by the relying domain. Digital agent identity identifies the software role or family, deployed version and running instance at the needed granularity. The organisation, agent instance, wallet operator, credential issuer, credential subject and presenting holder are separate roles. The W3C credential model distinguishes issuer, holder and subject; verification protects the credential's stated provenance and integrity without establishing the truth of every claim. Authorisation requires an accompanying framework [20].

A decentralised identifier (DID) can identify an organisation, workload or other subject and associate verification methods with a controller. DID control alone does not establish an organisation's registered identity or a representative's mandate [21]. The proposed profile therefore connects an accepted organisational identity record to authorised representatives or services, and from them to scoped agent delegations. Other identifier and credential mechanisms can serve this interface when they provide the required bindings. Neither the mathematics nor the execution contract requires every actor to use a DID.

The delegation path binds the principal, delegate instance or permitted workload class, allowed operations and resources, purpose, limits, validity, revocation and permitted redelegation. A verifier checks every required link and each issuer's authority to make that grant. A child delegation may only exercise scope supported by its parent and local policy. An organisational credential establishes neither unlimited delegation nor permission to use every key held in the same wallet. Changes to the running instance or deployed version must preserve the approved binding or trigger a new decision. Credential status, root-authority revocation and assurance invalidation remain distinct checks. Their freshness and failure behaviour must fit the continuation's timing model. Rotating a key or wallet does not reset the episode.

Here a **business wallet** is the organisation-governed service that manages credentials, presentations and access to signing keys. An **agent wallet** is a restricted credential/key interface available to a workload under that organisation's policy. These are functional roles for this architecture. They do not assert conformance to a specific regulated wallet profile. An agent wallet can be a scoped partition of an enterprise service; it need not be a separate application, ledger account or independently controlled key store for every agent.

Organisation–wallet binding must identify the represented principal, wallet operator and authorised key-management service. Agent–wallet binding adds the instance or workload identifier, credential scope and authenticated calling context. For this profile, possession proofs bind presentations to the recipient and transaction challenge, and the delegated credential connects the proving key to the represented subject. OpenID for Verifiable Presentations (OpenID4VP) supplies presentation mechanisms with verifier and nonce binding when holder binding is used; its broader protocol also permits explicitly requested presentations without holder binding [22]. Our profile requires the binding needed by the transaction and does not infer mandate from key possession. Shared organisational root keys should remain outside an agent’s direct control. Separate instance credentials support targeted revocation, rotation and attribution without automatically withdrawing every other worker’s authority.

A **verifiable evidence graph** organises the claims on which those decisions depend. In this design its typed nodes include principals, instances, delegations, contract versions, deployment configurations, observations, assessment results, model commitments, leases, ledger entries and effect receipts. Typed edges record representation, authorisation, provenance, derivation and dependency. The PROV Ontology (PROV-O), a provenance vocabulary, provides terms for entities, activities, agents and provenance relations [23]; its term “agent” is a provenance role and need not mean an autonomous AI agent. Our graph adds the task-specific authority and assurance semantics.

Verification applies to the relevant claims, edges and versioned evidence slice, not merely to possession of a graph. Validation records which claims and relations are authenticated by accepted issuers and which follow from verified computations. The verifier checks semantic types, authority scope, freshness, status, conflicting claims and the declared dependency coverage. A missing edge cannot silently become evidence of no dependency. Two reports derived from one sensor retain that common source; graph multiplicity does not create statistical independence. Dependency edges are not themselves probabilistic kernels; their mapping into the joint model requires justification. A change to a relied-upon node identifies affected decisions for re-evaluation, subject to Section 6’s scoped reuse conditions.

Access control and selective disclosure can expose the evidence slice needed by a partner while protecting unrelated records. The assurance calculation still requires a sufficient information interface under Section 4; hidden evidence selection can change the posterior. A graph commitment fixes a chosen version, not completeness or truth of the underlying world. Paper 3 specifies private evaluation over declared complete views, including required prohibitions and adverse findings, with change-sensitive admission checks [14]. Paper 2’s probability theorem retains its own information, model and execution assumptions independently of that protocol.

14.7 Architectural levels and placement of wallets and decision modules

The mandate–assurance intersection is a decision rule that can be instantiated at several scopes. An organisation can constrain a task, a task supervisor can constrain a joint continuation, and a gateway can enforce the exact next effect. Each instance must state its action universe, represented principals, evidence scope, harm constraints and accounting responsibility. A per-agent decision cannot replace a task-level check for interactions. Rechecking an issued lease at several gateways does not create several independent risk allowances or, by itself, require another debit; a newly issued continuation follows Section 8.

Table 5 separates functional responsibilities from deployable components. The **eligibility evaluator** is a logical function: mandate evaluation supplies one predicate, and the assurance evaluator supplies the supported continuation and risk test. Their conjunction yields a context-bound decision. The protected gateway is the policy enforcement point for actual effects. This preserves

the distinction between decision and enforcement in the ZTA foundation of Section 2.3. A wallet supplies identity and evidence services to this flow; it does not become the risk estimator or enforcement boundary merely because it stores the signed result.

Table 5: Functional architecture. Components can be colocated, but their authority, evidence and enforcement responsibilities remain distinct.

Functional responsibility	Main purpose	Typical components	Bound output
Organisational governance	Set representation, trust and delegation rules	Principal administration; business wallet; issuer and status services	Identity and scoped mandate records
Transaction coordination	Agree obligations and assess joint compatibility	Contract negotiator; peer-assertion verifier; joint assurance or proved composition	Versioned transaction and continuation contract
Local assurance decision	Test local authority and continuing exposure	Credential verifier; evidence graph; mandate and assurance evaluators; protected account	Eligibility decision and assurance lease
Effect enforcement	Mediate the actual permitted operation	Gateway; API or actuator adapter; replay and version checks	Dispatch and acceptance records
Observation and closure	Update evidence and resolve pending exposure	Monitors; queue tracker; containment controller	Observations, effect receipts and closure evidence

An organisation normally operates its business wallet near its identity and governance services. Agents obtain narrowly scoped access through workload authentication, a wallet API or a protected local service. The assurance evaluator and account belong in a trust boundary that the proposing agent cannot rewrite. An enforcement point sits at every covered path to an external effect, including cross-organisation APIs and local actuators. A data-space connector can host relevant adapters, but its identity handshake or transfer agreement does not replace the action-bound lease and effect checks. Components may share a process only if that deployment preserves the required separation of authority and protection from the proposer.

The end-to-end flow is: establish the principal and instance bindings; verify scoped delegation and the peer’s accepted evidence; bind a transaction contract and local continuations; evaluate mandate and assurance at the relevant scopes; debit the protected account and issue the lease; then recheck and enforce at effect commit. Observations and typed receipts update the graph and joint state for the next decision. The same flow operates on both sides of an independent-MAS transaction. Wallets authenticate and present; evaluators decide; gateways enforce; monitors establish what became observable. A successful wallet exchange is therefore an input to runtime assurance, not its completion.

14.7.1 Micro meso and macro scopes

We distinguish three scopes at which the functions in Table 5 operate. The micro scope concerns an instance and its exact effects. The meso scope concerns an interacting task, team or workflow. The macro scope concerns organisational governance and agreements between organisations.

These are proposed definitions for this architecture. They describe responsibility and decision scope, not three independently certified safety layers. MOISE+, an extension of the Model of Organization for Multi-agent Systems (MOISE), distinguishes organisational structure, task organisation, and the permissions and obligations connecting them [24]. This supports keeping coordination and authority explicit; the runtime probability argument remains the separate construction of Sections 4–8.

Table 6: Proposed scopes for the functional architecture. A scope need not correspond to a separate process, server or legal entity.

Scope	Controlled object	Placement of components and state
Micro	Agent instance and exact operation	Scoped identity and wallet interface; local observations; gateway at each covered resource. A protected evaluator may serve several instances.
Meso	Task, team or bounded workflow	Task coordinator; joint assurance service; dependency and evidence state; persistent episode account; containment coordination.
Macro	Organisational policy and federation agreements	Business wallets; delegation and evidence policies; task limits; cross-domain contracts; oversight and intervention authority.

One organisation may operate many meso-level teams, while one meso-level workflow may span several organisations. In the latter case, each principal retains its own authority root and business wallet. Section 14.5’s contracts connect their obligations. A cross-company coordinator acquires only the authority granted through those interfaces. The joint episode, rather than the organisational chart, determines the scope of the probability claim. A macro-level portfolio claim would additionally need its own harm definition and aggregation argument.

The protected assurance service must receive information sufficient to represent interacting effects. A local worker cannot establish a joint bound while omitting a shared provider or remote queue. Raw observations need not all be centralised, but summaries or interface bounds must preserve the inference conditions. In particular, private telemetry used to choose a task, partner or new worker remains subject to Section 4’s information restriction. An authenticated child approval alone does not establish information sufficiency.

14.7.2 Coordination patterns and placement rules

Hierarchical coordination assigns subtasks through a coordinator. A nested or holonic organisation lets a team provide a bounded service to a larger team while managing its internal workers. Contract-based federation connects independently governed teams through explicit obligations. For the proposed deployment, hierarchical or nested coordination can organise work within each domain, while federation preserves separate authority across domains. Shared dependencies may cross all these groupings and must remain represented in the evidence and joint model. The choice of coordination pattern does not establish a distributed composition theorem.

A coordinating LLM remains a proposer unless its implementation separately satisfies the protected controller requirements. Its position in the task hierarchy grants no ability to rewrite the assurance service, ledger or policy. Place enforcement at the controlled API, data-transfer or actuator boundary. NIST SP 800-207A describes application and service identities, gateways and proxies for granular access enforcement across cloud environments [25]. An adapter implementing this paper must additionally bind the actual effect to the admitted continuation. Higher-level

policy can be installed locally with explicit versions and validity; every operation need not contact every governance service. Local decisions still require evidence freshness and failure responses compatible with the covered continuation.

Business wallets remain under organisational key-management and representation controls. A meso-level service can present task and contract claims through delegated wallet access. A micro-level instance receives only its permitted credential or signing interface. This arrangement does not require a separate full wallet installation for every worker. The evidence graph links the organisational mandate, task contract and instance decision to their common dependencies and exact effects. The protected evaluator and account may be physically distributed only where their state and commit behaviour preserve the logical contract.

Allocation of a ceiling, reservation of capacity and permanent risk debit have different meanings. Macro governance may allocate task ceilings, and a meso-level service may reserve capacity for proposed workers. Concurrent reservations must not duplicate available capacity. Under Section 8, an issued lease covers the complete admitted joint continuation through containment and receives its permanent debit. Checking that lease at several levels does not debit it again. Independently issued child leases require a justified composition and accounting construction; summing child allowances is insufficient by itself. Releasing an unused reservation requires proof that it cannot still be consumed. Retirement or successful completion never refunds an issued risk debit in the present construction.

14.8 Dynamic agent creation and task scoped lifecycles

A task-scoped ephemeral agent is a running instance created for a bounded purpose and retired when its permitted participation ends. A one-time task can require several calls, observations and retries. It does not imply exactly-once execution. Dynamic creation applies the covered join and fork rules of Section 6 and the accounting discipline of Section 8. For a spawn operation in the declared action universe, admission requires

$$a_{\text{spawn}} \in \mathcal{A}_t^{\text{mandate}} \cap \mathcal{A}_t^{\text{assurance}}.$$

The mandate test includes permission to create a worker and delegate the requested scope. The assurance test covers the membership transition, provisioning effects, dependencies and resulting continuation. Copying sensitive context to the new worker can itself create exposure. A new instance must therefore enter through a controlled lifecycle, rather than receiving tool access before its admission is recorded.

1. **Request and authorise creation.** Bind the task, represented principal, requesting agent, approved worker class, tools, destinations and lifetime. Specify whether further spawning is permitted and bound its scope. A child's delegation must be supported by the issuer's authority and the receiving domain's policy.
2. **Provision a restricted instance.** Bind a fresh instance identifier and authentication context to the accepted configuration. Grant only the required wallet interface and credentials. Keep organisational root keys and control state outside its authority. Any provisioning or join exposure must belong to an already issued and permanently debited continuation before it occurs. Restricted staging alone is not an assurance argument.
3. **Admit the worker.** Apply the covered join or fork transition before new dependent effects. Retain common causes, current evidence and pending work. Before enabling task operations, verify the current continuation and protected instance and accounting records, or issue and debit

a supported replacement. Mere rechecking adds no debit. Activation must use coherent current authority, membership and assurance state.

4. Enforce operations. Check the current mandate and assurance at each covered effect. Bind the payload, destination and durable logical operation slot. A single-use effect needs protected consumption state at the enforcing boundary; expiry alone does not prevent replay. Retries and replacement workers reuse the existing slot and follow the resource’s stated idempotency and acknowledgement semantics. Section 13 checks replay for a fixed identifier universe; a production slot-allocation protocol must separately prevent re-identification of the same one-time effect [14].

5. Retire and close exposure. Remove permission for new ordinary task effects and withdraw instance access. Preserve authorised containment and protective actions. Retain pending operations, evidence and any predictive residual state until exposure closes or the charged tail applies. A protected service keeps this responsibility if the parent agent fails.

Workload identity, instance identity and task authority must be distinguished. The Secure Production Identity Framework for Everyone (SPIFFE) allows a workload to comprise several running instances and assumes sufficient isolation to protect issued credentials [26]. A workload identity therefore need not identify one temporary agent run or establish its mandate. The proposed profile adds the instance and task bindings required by Section 14.6. The evidence graph records who requested and authorised creation, which configuration ran, which delegation applied, and which episode, lease and receipts cover its effects. Destroying the process or its key must leave those records available to the authorised assurance and audit services.

Creation limits should cover concurrent workers, total creations within the bounded task, creation rate, redelegation depth, resource use and outstanding operations. These limits constrain expansion but do not replace joint assurance. Forks and replacement instances retain the episode reference and permanent debits. Missing receipts remain unresolved exposure. Revocation prevents new operations within the contract’s enforcement timing; it does not retract accepted remote work. Limits and parent-failure handling must be enforced outside the agents whose behaviour they constrain.

For example, A creates a temporary worker to release design chunk 17 to B’s named simulation job. Its delegation binds that payload, destination and operation. If the worker crashes after dispatch, a replacement reconciles the existing operation record before acting. A missing acknowledgement does not establish that release failed. B separately controls receiving and processing under its local contract. The replacement may observe and help close the same episode if authorised, but receives no fresh risk allowance merely because its identity is new.

An assessed worker template can supply reusable evidence about configuration and control coverage. Each instance still requires current identity, delegation, context and admission checks. A fresh instance within a covered worker class may use Section 6’s transfer rule. Arbitrary generated code, new tools or unmodelled dependencies require an additional coverage argument or a supported restriction. These lifecycle requirements specify an implementation of the existing bounded construction; they add no claim that unrestricted self-organisation or every generated agent is safe.

14.9 Operational alignment and the claim that can be verified

Enforcement of the intersection, under the stated assumptions, supports a limited form of operational alignment. Executed actions conform to compatible mandates, and their admitted joint continuation satisfies the model-based risk test. The task contract must state whose objectives count, how conflicts are resolved, who may revise limits and who bears each obligation. No agent may increase an allowance or change a harm definition merely to complete its task.

An empty intersection requires an authorised, covered alternative or the specified protective continuation.

This does not establish that the principals’ goals are mutually beneficial, that a harmful goal was excluded from the mandate, or that all affected people and organisations were represented. Collusion, specification gaming, omitted externalities and incentives need separate analysis. Paper 1 supplies part of the authority and incentive framework [1]; Paper 2 derives a conditional bound for covered exposure; Paper 3 specifies private verification and current reliance on bounded claims [14]. Their combination supports inspectable operational controls only under the respective assumptions, while broader AI alignment remains a distinct research and governance problem.

The research finding is therefore a boundary condition: organisational federation preserves the present safety argument only when its implementation preserves the analysed joint episode. Local approvals become useful inputs to that argument. They do not establish global safety through aggregation alone.

15 Deployment patterns operational oversight and verified containment

The same eligibility rule can support different operating arrangements, but responsibility for its inputs and enforcement changes with deployment. This section develops worked design cases for a frontier provider, independently operated models and cloud–edge systems. They connect Section 14’s architecture to implementation choices; they are neither reports of field trials nor extensions of Theorem 2. Each case requires a named harm, bounded effects and evidence sufficient for the declared model. A general-purpose model evaluation does not supply those conditions for every customer application.

15.1 Deployment taxonomy and governance responsibilities

AI governance assigns responsibility for objectives, acceptable risk, deployment and intervention. Trustworthy AI concerns broader system qualities, including reliability, safety, security, transparency, privacy and fairness. NIST AI RMF 1.0 organises risk management through GOVERN, MAP, MEASURE and MANAGE, with continuing review across the lifecycle [27]. In this paper, **trusted AI** means justified reliance on an identified deployment for a stated purpose. It is a scoped relationship supported by evidence, not a certification label. AI safety concerns the named harms; operational alignment concerns the mandates and agreed constraints in Section 14.9. The eligibility rule provides one technical mechanism within these wider responsibilities.

Three deployment families determine who can observe and constrain execution. A **provider-operated service** places model serving, and possibly agent execution, within the provider’s boundary. An **independently operated deployment** places the runtime and application controls under a customer or other operator. A **hybrid cloud–edge deployment** combines remote proposals with local enforcement and protective responses. These families can realise different supervision patterns from Section 14.4. Deployment ownership does not establish whether supervision is logically joint or contract-based.

Model access is a separate dimension. A proprietary model can be supplied through an API or, where licensed, in a customer-operated environment. Available weights permit inspection or local operation only to the extent supported by the accompanying materials and licence. The Open Source Initiative (OSI) Open Source AI Definition additionally specifies freedoms and access to code, parameters and training-data information [28]. We therefore distinguish closed models,

open-weight models and systems meeting that definition. None of these labels determines the effectiveness of deployed controls. Figure 10 locates the effect boundaries across the deployment families.

Deployment ownership determines who can enforce the decision

Proposed architecture: model access is separate from execution ownership.

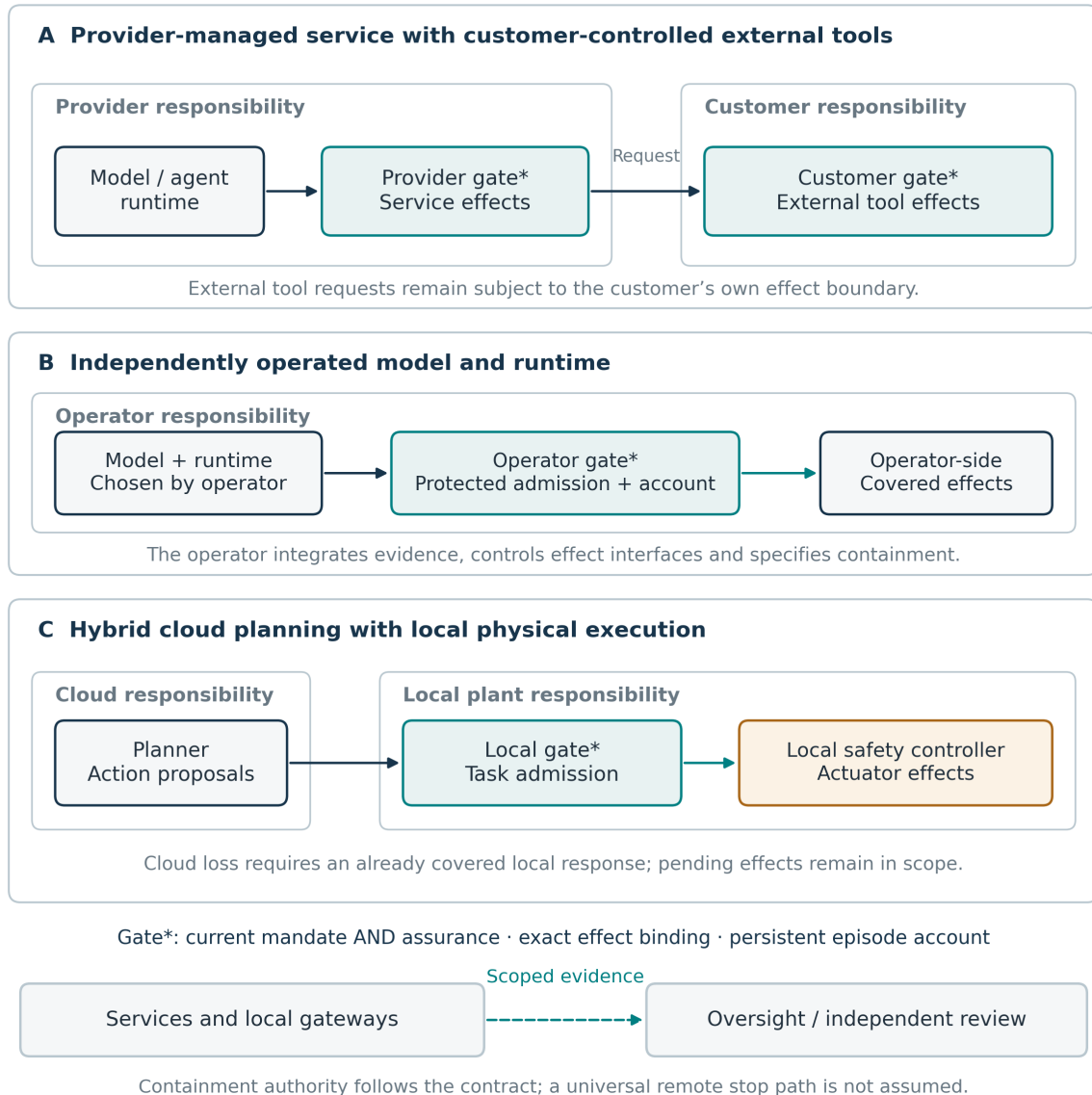


Figure 10: Proposed deployment responsibilities. Model access and execution ownership are separate dimensions. Each covered effect requires a protected eligibility decision and an enforcing gateway. Gates bind the exact continuation and use persistent episode accounting with permanent debits; gates sharing one episode require coherent accounting or a justified allocation protocol. Oversight receives scoped evidence; intervention authority follows the deployment contract. Hybrid operation requires a covered local response to communication loss while local protective functions remain available. These are reference architectures rather than validated deployments.

15.2 Worked case of a frontier provider and its customers

Consider a hypothetical large technology provider P supplying a frontier model API and a managed agent service to engineering companies A and B. Its agents analyse designs, book

compute and propose transfers to simulation services. The case uses the protected-data harm from Section 3. Additional harms, such as abusive tool use or unsafe equipment commands, would require their own specifications and evidence. Provider-wide AI safety is broader than this single application contract.

Public documentation supplies useful reference points. Google DeepMind’s Frontier Safety Framework describes capability-based assessment and mitigation processes [29]. Google Cloud’s Model Armor distinguishes inspection from blocking; its calling integration enforces a block verdict. It also describes single-turn screening without retained conversation history [30]. These sources illustrate release governance and runtime filtering. They do not document the episode supervisor proposed here. A filter’s confidence category is not a bound on accumulated first-harm probability.

Provisioning. A identifies its organisational principal and authorised administrators, then delegates a bounded engineering task to agent instances. P registers the permitted model/service revisions, tools, data destinations and relevant dependencies. The service agreement assigns incident contacts, observation access, change notification and containment duties. Sections 14.5–14.8 supply the contract, wallet, evidence and lifecycle interfaces. Customer credentials establish representation; they do not grant unrestricted access to P’s resources or B’s receiving environment.

Admission and execution. Suppose A’s agent requests release of design chunk 17 to B’s named job. The required principals approve their respective obligations in the common record of Section 14.2. A’s data policy, P’s service restrictions and B’s receiving authority apply where each controls the proposed effect. The joint assurance decision covers the selected continuation, provider dependencies, queue state and containment delay. A permanent debit precedes dispatch. The gateway compares the actual payload and destination with that decision at commit. An accepted booking supplies neither the release mandate nor the release assurance.

Temporary workers. A may request a managed worker from P for this single transfer. A’s mandate binds the task, while P admits and constrains the instance under its own service policy. The task-level supervisor retains the episode and pending queue across worker replacement. A worker’s expiry blocks further ordinary operations within the agreed timing; P’s protected service remains responsible for its containment obligations. B retains its own receiving gate. An API-only service leaves A responsible for the lifecycle of the workers it operates.

Two service modes require different implementation boundaries. In API-only provision, P controls its service endpoint but may neither observe nor mediate subsequent customer tool calls. A therefore needs its own protected decision and effect gateways. In the managed-agent mode, P can mediate the tools that it operates, while A and B retain gates over external resources they control. A customer-side gateway cannot constrain undisclosed onward processing inside P by itself. Such processing needs a supported provider contract, accepted evidence and a composition argument, or a narrower release policy. Responsibility follows actual effects rather than the location of the conversational interface.

Internal controls. The provider’s own coding and research agents create another application boundary. In the proposed design, they receive scoped repository and tool access, execute untrusted work in isolated environments, and pass protected review gates before changing serving infrastructure or safety controls. They cannot alter their own assurance service, episode ledger or intervention authority. Training and release evaluations address model behaviour; deployment controls address the permitted effects of running instances. AISI’s research agenda treats agents inside frontier developers’ infrastructure as a control setting and studies adversarial evaluation of safeguards [31]. The present paper specifies a bounded interface for such controls, not a proof against every form of deception or control subversion.

Cross-customer assurance. Customers may pursue different legitimate objectives under common service restrictions. Tenant isolation must cover identities, credentials, caches, tools and data paths. Shared model revisions, hosting services and monitoring infrastructure can create correlated failures across tenants. Separate task allowances therefore provide no automatic provider-wide bound. A fleet-level risk claim needs its own harm definition, exposure population and justified aggregation, including common causes. Per-tenant monitors can feed a provider incident process without treating reports from one common source as independent evidence. Model or tool changes require reassessment of the affected contracts before new effects rely on them.

Intervention. If evidence of B’s isolation becomes stale, the design prevents further unsupported release and invokes the continuation already covered for accepted work. If a shared dependency fails, the provider can widen the intervention to affected tenants or a service revision. Its scope and downstream effects must be covered. This case matters because one provider may serve many applications while controlling only part of each end-to-end workflow. Clear responsibility and evidence boundaries make that limitation inspectable.

15.3 Closed models and verifiable assurance assertions

A closed model can participate when accepted evidence supports the relevant deployment claim. Source disclosure and assurance are different questions: open code enables inspection, while a closed service may permit confidential assessment or externally observable tests. Neither access mode establishes the complete runtime argument. We propose a typed assertion profile for closed models, also usable for open deployments, that extends the transaction assertions in Section 14.5.

The profile binds the provider and authorised issuer to the model/service revision, routing policy, deployment configuration, available tools and intended operating conditions. Where an API alias can select several backends, the claim must cover that family or identify the actual revision used through an accepted binding mechanism. A claimed model name alone leaves substitution unresolved. The assertion also states its validity, status mechanism, change-notification duties and the evidence needed to continue relying on it.

A TEVV assertion identifies the assessed claim, threat model, test population, protocol, evaluator and access level, results, uncertainty, known failures and limits of transfer to deployment. NIST calls for documented methods, deployment-relevant evaluation, assessment of limitations and production monitoring [27]. The fields proposed here are an implementation profile, not a NIST credential schema. A benchmark result remains evidence about its tested conditions; converting it into an episode bound requires the model and inference arguments of Sections 4–8.

Table 7: Proposed assurance assertions for closed and open deployments. The rows concern different claims and are not interchangeable certificates of safety.

Assertion type	Required binding in the proposed profile	What remains to be established
TEVV result	Evaluator, tested revision, conditions, methods, results and limits	Relevance to the actual task and environment
Safety control configuration	Mechanism, version, coverage, enforcement point and failure response	Correct installation and absence of bypass paths
Control activation	Measured deployment, active policy, evidence time and status	Continued validity at the operation

Table 7 (continued)

Assertion type	Required binding in the proposed profile	What remains to be established
Action admission	Exact operation, mandate epoch, lease and permanent debit	Conformance of the actual execution
Effect or containment receipt	Resource, observed event, time and unresolved work	Closure of the complete exposure

A provider-signed assertion attributes a statement. An independent assessment adds evidence within its declared scope and access. An external API test observes behaviour under tested conditions; identifying the internal build or routing needs additional evidence. Runtime attestation addresses measured state under its trust assumptions. An end-to-end argument must connect these claims to the deployed effect boundary. An assertion that a filter is installed, for example, does not establish that it was enabled, invoked for this request or enforced by the gateway. Control owners must identify these separate states and the records that support them.

The evidence graph links assessments and control claims to deployment versions, mandates, observations, leases and receipts. Wallets can present authenticated claims and status information; the relying evaluator decides whether they satisfy its evidence policy. Confidential evaluation or a proof of a specified computation may reduce disclosure. It cannot replace missing evidence of observation quality or remote execution. Section 4’s information restriction also remains: private telemetry used for proposal selection must enter the declared inference interface or a justified selection model. Proprietary implementation is not an exemption from that condition.

In the worked case, P might present an assessment for revision R, a claim that a specified data-loss control is enabled, and a receipt identifying the actual tool operation. A checks whether all three concern the same accepted deployment and whether their scope supports its release contract. A silent backend change, expired control claim or unsupported information interface requires reassessment or a covered restricted response. It must not be converted into permission by treating absent evidence as a favourable score.

15.4 Independently operated open models

An organisation can deploy an open-weight or open-source model on its own infrastructure and apply the same eligibility rule. Local operation gives it control over version selection and integration. The proposed baseline pins the model, tokenizer, adapters, inference runtime, agent framework, tool definitions and policy configuration in a deployment manifest. The organisation’s business wallet manages representation credentials and protected signing interfaces. Agent instances receive narrow delegations. A separate protected service evaluates mandate and assurance and maintains the episode account; gateways mediate all covered effects.

Evidence of proper operation has several layers. Signed manifests and build provenance identify supplied artifacts. Code review and finite or formal checks address the implemented decision logic. Adversarial integration tests challenge bypass paths, replay, stale decisions and accounting rollback. Runtime observations establish which relevant events became visible. Remote attestation can bind measured components to reference values under an explicit trust model. The Remote Attestation Procedures (RATS) architecture in RFC 9334 distinguishes evidence, appraisal and relying-party decisions, and recognises that state may change immediately after evidence generation [32]. A platform measurement covers a model, accelerator runtime or tool configuration only when the measurement chain actually includes it.

For temporary local workers, a deployment controller can instantiate an assessed template in a restricted runtime, bind the instance to its delegated wallet interface, and enable tool access only after admission. The episode account, operation-consumption records and containment service must outlive worker processes and survive their replacement. Template evidence can be reused within its scope; each activation still checks current authority, configuration, dependencies and the proposed continuation. These are the independent operator’s obligations under Section 14.8.

For example, A can replace the hosted proposer with a locally served model while retaining the same protected release adapter. This preserves the architectural interface, but it does not by itself preserve the probability model. A changed proposer may use different private information, tools or action-selection behaviour. The operator must show that the declared information contract and all relevant kernels still apply, or revise the assurance construction. Model openness supplies neither that demonstration nor field calibration.

Practical challenges include privileged administrators, alternate credentials, unrestricted egress, unmeasured extensions, incomplete logs and compromised monitoring. The trust model must identify who can alter the supervisor or bypass the gateway. Controls against an untrusted agent do not automatically constrain an administrator who controls the entire host. A partner may require independently assessed deployment controls or condition access to its own resources on fresh evidence. A model publisher cannot ensure that every independently modified copy retains a supervisor or accepts a remote stop request. The defensible claim therefore concerns an identified deployment and its mediated resources, not every copy of the model.

15.5 Hybrid cloud and edge execution

In a hybrid deployment, a remote model proposes a bounded operating plan while a local supervisor and gateway control physical effects. Consider a battery operator receiving a proposed power schedule from a cloud agent. The local mandate identifies the asset, permitted service and operating limits. Assurance requires the relevant equipment state, sensor quality, dependencies and response timing. A local protective controller handles the specified communication-loss or unsafe-state response. This is an architectural example; the data-release experiment does not establish battery safety.

The cloud model can be closed or open. Its access mode does not remove local timing requirements. NIST’s OT guidance recognises independent safety systems and orderly shutdown needs [18]. Disconnecting an agent or its wallet must therefore leave required protection functioning. The admitted continuation must cover message delay, partitions, queued commands and clock uncertainty. Renewal after reconnection preserves the episode and pending state. Independent-MAS participation by an aggregator or maintenance provider adds Section 14.5’s compatibility obligations.

15.6 Internal monitoring and independent oversight

Operational monitoring supplies observations for admission and protective responses. Internal compliance monitoring examines whether approved mandates, controls, reviews and incident procedures are followed. Independent assessors challenge the assurance argument and reproduce selected checks. Regulatory authorities exercise their applicable legal powers. These functions differ in access, timing and authority; an audit dashboard is not an inline execution gate.

NIST AI RMF identifies continuing monitoring and defined responsibility, independent assessment, mechanisms to disengage or deactivate systems, and post-deployment response and recovery [27]. AISI studies control evaluations and the transfer of safeguards from test environments to

deployment [31]. Its role should be described accurately: the UK government’s account identifies AISI as a research institute, not a regulator [33]. The European Commission separately describes the AI Office’s model-evaluation powers and its ability to request measures including restrictions on public availability [34]. These are jurisdiction-specific oversight arrangements, not a universal alignment-certification service or a technical switch over every deployment.

In the proposed architecture, control monitoring reconciles gateway records with resource receipts and the protected account. It checks decisions against current mandate and policy versions, searches for uncharged effects, tracks missing observations and identifies overdue containment. Compliance staff receive scoped evidence views and escalation records. External assessors receive the evidence and test access needed for their stated claim, subject to confidentiality and access rules. Tamper-evident records protect recorded history; completeness requires separate reconciliation and observation coverage. Monitoring should preserve customer separation and minimise retained sensitive content.

A monitoring service also creates a dependency. Its failure, compromise or excessive delay must have a declared response, and an agent must not be able to silence it to expand executable scope. Human escalation requires named authority and feasible response times. Immediate protective actions can be pre-authorised; consequential overrides and restart decisions require an accountable process. External findings or lawful directions can enter that process through authenticated channels. The local intervention service still needs defined authority, scope and effect semantics. Section 15.7 specifies the corresponding kill-switch protocol.

15.7 Kill switch for verified containment and controlled restart

A **kill switch** is a protected intervention mechanism that restricts the covered effects of an identified deployment. We specify it as a containment protocol with observable completion conditions. It can be triggered by a covered monitor condition, authority revocation, lease expiry, or an authenticated decision by an authorised operator. NIST AI RMF MANAGE 2.4 calls for assigned responsibilities and mechanisms to supersede, disengage or deactivate systems [27]. Our protocol supplies a proposed execution interface for that responsibility; it is not a model-wide guarantee or a new theorem. The first step prevents new ordinary task effects at protected gateways while preserving authorised containment and protective actions. It uses current authority and lease versions. The intervention service withdraws relevant agent tool credentials and stops or isolates the runtime, while retaining its own scoped authority to complete the response. It also invokes the agreed response for work already accepted by remote systems. Closure requires the declared evidence that exposure ended, or the residual tail already included in the charged continuation. The protocol can target an operation, agent, tenant, dependency or service revision, with wider scopes justified by the incident and its consequences.

The protocol records four milestones: the intervention request; blocking of new ordinary effects; local runtime containment; and closure of remote or physical exposure. Each record identifies its scope, responsible service, policy version, observed time and unresolved work. Milestones may overlap or occur in a different physical order, but closure must satisfy the declared exposure criterion. An acknowledged command proves only the acknowledged event. A communication partition leaves remote outcomes unresolved. The risk calculation may use an explicitly charged residual-tail bound when its assumptions hold; that bound is not a receipt confirming remote termination. Kubernetes documents a concrete control-plane limitation: force deletion of a Pod’s API object does not confirm termination of its running workload [35]. A lost acknowledgement similarly leaves remote work unresolved in this paper. Section 7’s containment delay and queue state represent those gaps. Network isolation or process termination may be useful actions, but their safety depends on the application and the covered continuation.

The kill switch must remain usable when the proposer, coordinator or its wallet is compromised or unavailable. Its protected service needs independent scoped authority, access to enforcement points and an observation path sufficient to assess completion. Independence here is an architectural requirement under the declared failure model, not an assumption that all common causes disappear. In a cross-organisation workflow, each organisation controls its own resources and honours the agreed intervention contract; no participant obtains an unrestricted switch over its partners. Micro-level intervention can restrict an instance or operation, meso-level intervention can contain a task, and macro-level authority can widen the scope to affected tenants or service revisions. The selected scope and response must be covered by the continuation.

For cyber-physical systems, cutting power or communication can itself create harm. The protocol must preserve local protective functions and use the assessed transition to a safe operating condition [18]. For agent services, it must also prevent unapproved respawning, retries or key replacement from restoring ordinary access while containment is active. Section 14.8 therefore binds worker replacement to the existing episode and intervention state.

The intervention path must be protected from the proposing agent and tested under the declared failure model. Authentication, scope limits, separation of duties and records of emergency use reduce the risk that the stop mechanism becomes an unauthorised denial-of-service channel. The trigger’s information, detection delay and response dynamics must enter the declared history and charged continuation for the quantitative bound to apply. NIST AI RMF MANAGE 4.1 places recovery within post-deployment risk management [27]. In the proposed protocol, restart requires an authorised decision after the fault, current mandate, intervention state and affected evidence have been reviewed. Any resumed continuation must pass a new eligibility decision and receive its required debit before producing effects; clearing an alarm is insufficient. Restart preserves unresolved effects and prior permanent debits; a new process or key does not create a fresh risk allowance. If an incident exceeds the analysed model, emergency response remains necessary, but the theorem cannot be claimed for the resulting unmodelled execution.

15.8 Deployment evaluation and intervention tests

Deployment evaluation should measure detection delay, time to block new effects, local termination time, remote containment time and remaining exposure. It should also report useful work, false refusals, missed hazards, monitoring cost and human response capacity. Tests should include partitions, stale assertions, backend substitution, model and tool updates, compromised telemetry, ledger rollback and shared-provider failure. Kill-switch tests should additionally cover an unavailable coordinator, forged intervention requests, automatic worker restart, lost stop acknowledgements and a failed primary enforcement path. Verify that protective functions remain available and that delayed closure stays within the admitted response model. The present synthetic and controller checks cover selected timing and state distinctions; they do not validate this complete deployment protocol. Demonstrating that a score was computed is weaker than showing that every covered effect passed the required gate. Section 16 turns these cases into staged validation obligations. The proposed patterns make implementation responsibilities explicit while leaving empirical model coverage and broader AI alignment as separate research and governance tasks.

16 Applied AI safety and implementation obligations

16.1 A practical prototype boundary

An applied AI safety prototype can connect two principals, a bounded worker pool, a booking service, a release gateway, a provider simulator and one authoritative accounting service. Begin with an explicit operation menu and authenticated evidence. Measure the boundary between the gateway’s dispatch and the remote effect before adding a data-space connector or private proof adapter. The baseline requires authenticated evidence records; it does not require a zero-knowledge layer. Signature validity alone does not discharge observation quality or inference assumptions. Implement the shared-supervision pattern first, then separate the parties’ gateways while preserving the same logical contract. The wallet, evidence and lifecycle services in Sections 14.6–14.8 supply authenticated inputs; local evaluators and gateways retain responsibility for admission and effects. Before testing independent-MAS federation, specify the contract-to-local-continuation mappings and the joint property being claimed.

The lease record should bind the episode and owner; represented principal and instance; exact action or bounded policy; harm definition; model and evidence commitments; dependency scope; authority epoch; assurance generation; configuration and policy versions; operation expiry; containment policy; and permanent harm-specific debit. A record needs units and time semantics, not just a score and a timestamp. Principal, agent family, process instance and session remain different identifiers. For cross-organisation execution, add the required-principal set, each party’s mandate decision, the common continuation commitment and the local gateway obligations. Bind policy installation and acknowledgement status to that record; a proposal identifier alone cannot resolve mixed policy versions.

Table 8: Implementation obligations. A successful computation in one component does not discharge the others.

Component	Concrete responsibility	Required evidence before deployment
Wallet and mandate adapter	Bind principal, instance, proving key and scoped delegation	Issuer acceptance, subject/key substitution, status and delegation tests
Peer assertion verifier	Check contract, local witness and permitted reliance	Stale readiness, replay, role mismatch and unsupported-claim tests
Observation adapter	Authenticate, timestamp and interpret reports and missingness	Labelled data, sampling assumptions and corruption tests
Joint estimator	Retain shared causes, hidden harm and pending effects	Filter and lifecycle coverage checks
Lease evaluator	Bound the exact continuation through containment	Model coverage, numerical enclosure and timing measurements
Episode ledger	Keep allocations across retries, renewal and workers	Coherent shared accounting and roll-back/race tests
Execution gateway	Bind actual payload and current context at commit	Substitution, replay, expiry and bypass tests
Remote-effect adapter	Track acceptance, idempotency and unresolved outcomes	Queue and acknowledgement semantics under faults

Observation evidence, decision records, commands and confirmed effects should remain distinguishable in the audit trail. “Stop sent” is not “stop effective”; “request accepted” is not “effect completed.” If a new provider is outside the bank, the system needs a justified model extension or a restricted response whose existing assumptions still apply. A fail-closed gateway alone cannot erase autonomous exposure already in the environment.

A staged implementation should first reproduce one organisation’s joint worker supervision. Next, place the sender and receiver behind separately operated gateways, with a common record and accounted remote queue. Introduce bounded joins from a third organisation only after those interfaces are verified. This progression changes the enforcement boundary while keeping the harm definition explicit. It is a proposed validation sequence, not a report of completed integration.

16.2 Evaluation and incentives

Testing, evaluation, verification and validation should report useful work, refusals, harm and intervention delay together. Refusing everything can minimise exposure while failing the intended task. An evaluation should include misleading telemetry, shared-source duplication, private proposer information, forked requests, delayed acknowledgements and revocation during pending work. The first-harm measure also needs an incident response: lower incremental first-harm probability after suspected prior harm must not be interpreted as restored health.

Cross-organisation tests should exercise simultaneous spending of one allocation, conflicting continuation approvals, different policy versions during renewal, private telemetry that affects proposal selection, partitions during revocation and reconnection with pending effects. A swarm test should include a harm created only by three-way interaction, as well as shared-service failure and a departing agent’s unresolved operation. Record whether every actual effect remains within a charged, jointly covered continuation. Aggregate the results by failure class; successful signature verification is only one part of that evidence. The contract-federation tests should also swap a credential subject or instance key, widen a child delegation, replay an operation within one transaction, clone a wallet interface and present a stale debit reference. Evidence-graph tests should omit a required dependency, duplicate one source under several issuers, alter a versioned claim and hide information used for partner selection. These are proposed validation cases; the existing finite checker does not establish their results.

Hierarchy and lifecycle tests should exercise concurrent creation against one reservation, stale parent authority at activation, excessive redelegation, a parent crash with live children, and a replacement after dispatch but before acknowledgement. Include replay after key rotation, attempted ledger rollback, template or tool substitution, and creation selected using undisclosed telemetry. Verify that each instance enters through a covered transition, every admitted effect remains charged, and retirement retains pending exposure. Test the protective response when spawning or monitoring exceeds its declared timing. Report these as new validation cases; the synthetic results and finite checks in Sections 12–13 do not yet establish them.

The action rule changes the feasible choices in Paper 1’s game. Shorter leases, probes and assurance computation have costs. A worker may prefer a faster fallback or attempt to route work through a cheaper evidence path. The controller can enforce the declared scope and account; it does not establish that every authorised choice is strategically optimal or socially adequate. A cost-aware incentive analysis must include the actual assurance policy and its fallback outcomes. This paper does not prove a new equilibrium result.

A promising next deployment experiment would measure a real release adapter’s queue states, containment latency and evidence quality under controlled faults. Apply Section 15’s hosted and independent proposer arrangements to the same mediated action family, after checking

their information interfaces. Compare provider-only assertions, independently assessed evidence and deployment-bound control records as separate evidence conditions. Evaluate unsupported claims as admission failures rather than assigning them favourable risk values. The hybrid case additionally needs measured local responses under communication loss. A subsequent algorithmic study should compare the scoped update with efficient published shielding methods under matched observation and computation budgets. The present full-history comparator isolates repeated inference cost; it is not a substitute for that comparison.

16.3 Contributions and limits

The renewal target is met for a containing finite model bank and a permanent full-continuation account. The membership target is met by a joint transfer and sufficient marginal-reuse criterion. The enforcement target has an inductive refinement argument and finite checks. The information-limit target is an application of established indistinguishability reasoning. Monitoring value is examined through finite policy comparisons and a binary probe calculation; globally optimal sensing, online hidden-rate calibration and field effectiveness remain open.

The most important limits follow directly from the proof. The bank must contain a complete model, including lifecycle, observation and containment semantics. The proposer must respect the information scope or supply a modelled selection signal. Initial, intervening and final exposure must be covered. Numerical bounds and effect mediation must be sound. Removing any of these conditions can invalidate the operational guarantee while leaving a credential or proof syntactically valid.

The cross-organisation analysis adds an application contract and a boundary for the current results. Shared logical supervision can represent multiple principals when its existing assumptions hold. Distributed policy installation, compositional local certificates, confidential evidence interfaces and scalable swarm abstractions remain verification tasks. Their implementation must preserve joint exposure and information semantics; increasing the number of signed approvals does not establish that preservation. The added identity and wallet taxonomy specifies who can present which claims and where decisions are enforced. The micro, meso and macro mapping locates those functions across instance, task and governance scopes. Task-scoped creation and retirement preserve the existing episode through bounded membership changes. These extensions supply implementation obligations, not an additional theorem of distributed safety or a validation of a wallet product.

The deployment cases locate the same obligations in frontier services, independent runtimes and cloud-edge systems. Their TEVV and safety-control assertion profile distinguishes evaluation evidence, current configuration, admission and observed effects. NIST AI RMF and AISI's control research supply governance and evaluation context [27,31]; neither is evidence that the proposed implementation has been validated. These cases add deployment requirements, not a new probability theorem or a finding that a proprietary or open model is generally aligned.

For applied AI safety, the completed contribution is a small, testable connection between authority, uncertain evidence and controlled effects. It supports a concrete question at each decision: which exact action and continuation are both permitted and within the remaining declared allowance? The supplied code, counterexamples, controller traces and figures make that question reproducible. Production reliance additionally requires measured observation quality, actual containment behaviour and validation of the chosen harm specification. Governance must identify who accepts residual risk and who can act when those conditions cease to hold.

A Reproducibility and research status

The accompanying figure package includes all ten figures as editable SVG, vector PDF and high-resolution PNG, a combined review PDF, captions and build code. Its reproducibility directory contains the synthetic experiment and controller checker, configurations, raw CSV and JSON results, mathematical review and the completed research-gate record. The main experiment uses Python 3.12 with the NumPy and SciPy numerical libraries; the controller checker uses the Python standard library. Commands and recorded versions are given in each README. Run assertions enabled. The hierarchy and lifecycle design in Sections 14.7–14.8, Figure 10 and Section 15 are architectural and deployment extensions; they add no empirical observations to those experiments.

The analytical arguments received an internal mathematical review, and the executable checks were rerun. The code and data are supplied in the accompanying package; no public repository identifier is claimed. No external peer review, proof-assistant verification, production calibration or field deployment is claimed. The literature review is targeted and reviewed through 26 September 2026, not a systematic census. Preprints, published papers, technical specifications and working manuscripts retain their separate status below.

References

- [1] Stöcker, C. (2026). *Resource Bounded Program Equilibria under Cryptographic Commitments and Revocable Authority*. Companion Paper 1, working manuscript, series revision 28 September 2026. Author's research catalogue: <https://spherity.github.io/spherity-research/>. Companion PDF (same folder).
- [2] Pedro Santana, Sylvie Thiébaux, Brian Williams. (2016). RAO*: An Algorithm for Chance-Constrained POMDP's. *Proceedings of AAAI 30(1)*. <https://doi.org/10.1609/aaai.v30i1.10423>
- [3] Usama Mehmood, Sanaz Sheikhi, Stanley Bak, Scott A. Smolka, Scott D. Stoller. (2022). *The Black-Box Simplex Architecture for Runtime Assurance of Autonomous CPS*. *NASA Formal Methods*, pp. 231–250. https://doi.org/10.1007/978-3-031-06773-0_12
- [4] Hung-Jui Huang, Kai-Chi Huang, Michal Čáp, Yibiao Zhao, Ying Nian Wu, Chris L. Baker. (2021). *Planning on a (Risk) Budget: Safe Non-Conservative Planning in Probabilistic Dynamic Environments*. *ICRA 2021*; arXiv:2106.09127v1. <https://arxiv.org/abs/2106.09127v1>
- [5] Daniel Schneider, Mario Trapp. (2013). *Conditional safety certification of open adaptive systems*. *ACM Transactions on Autonomous and Adaptive Systems 8(2)*, Article 8. <https://doi.org/10.1145/2491465.2491467>
- [6] J. Tanner Slagel, Lauren M. White, Aaron Dutle, Cesar A. Muñoz, Nicolas Crespo. (2024). *A Verification Framework for Runtime Assurance of Autonomous UAS*. 43rd Digital Avionics Systems Conference; NASA document 20240007986. <https://ntrs.nasa.gov/citations/20240007986>
- [7] William Scarbro, Ravi Mangal. (2026). *Interval POMDP Shielding for Imperfect-Perception Agents*. arXiv:2604.20728v2; authors identify an extended EMSOFT 2026 paper. <https://arxiv.org/abs/2604.20728v2>
- [8] Haoran Yang, Nobuko Yoshida. (2026). *Generating Local Shields for Decentralised Partially Observable Markov Decision Processes*. *EPTCS 444*, pp. 1–10; *PLACES 2026*. <https://doi.org/10.4204/EPTCS.444.1>
- [9] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, Jasjeet Sekhon. (2021). *Time-uniform, nonparametric, nonasymptotic confidence sequences*. *Annals of Statistics 49(2)*, 1055–1080. <https://doi.org/10.1214/20-AOS1991>
- [10] Luko van der Maas, Sebastian Junges. (2026). *Learning Verified Monitors for Hidden Markov Models*. *Automated Technology for Verification and Analysis (ATVA 2025)*, *Lecture Notes in Computer Science (LNCS)*, vol. 16145, pp. 180–203. First online 26 October 2025. https://doi.org/10.1007/978-3-032-08707-2_9
- [11] Viraj Parimi, Brian C. Williams. (2026). *Risk-Bounded Multi-Agent Visual Navigation via Iterative Risk Allocation*. *ICAPS 2026*; arXiv:2509.08157v3. <https://arxiv.org/abs/2509.08157v3>
- [12] Guangxi Wan, Yongbo Xie, Yuqi Liu, Qingwei Dong, Qingxin Li, Hongfei Bai, Peng Zeng. (2026). *Generator-Independent Runtime Assurance under Partial Observation*. arXiv:2609.06036v1, preprint, 5 September 2026; draft v0.5. <https://arxiv.org/abs/2609.06036v1>

- [13] Guangxi Wan, Hualong Du, Yuqi Liu, Qingwei Dong, Qingxin Li, Hongfei Bai, Peng Zeng. (2026). *Observation Design for Certified Control Authority: Projection–Estimability Separation and Active-Face Equivalence*. arXiv:2609.05962v1, preprint, 5 September 2026. <https://arxiv.org/abs/2609.05962v1>
- [14] Stöcker, C. (2026). *Private Verification of Action Eligibility across Trust Domains*. Companion Paper 3, working manuscript, series revision 28 September 2026. Author's research catalogue: <https://spherity.github.io/spherity-research/>. Companion PDF (same folder).
- [15] Evangelos Haleplidis, Kostas Pentikousis, Spyros Denazis, Jamal Hadi Salim, David Meyer, Odysseas Koufopavlou. (2015). *Software-Defined Networking (SDN): Layers and Architecture Terminology*. IRTF RFC 7426, Informational. <https://www.rfc-editor.org/rfc/rfc7426>
- [16] Scott Rose, Oliver Borchert, Stu Mitchell, Sean Connelly. (2020). *Zero Trust Architecture*. NIST SP 800-207. <https://doi.org/10.6028/NIST.SP.800-207>
- [17] Eclipse Dataspace Components contributors. (2026). *Developer Handbook*. Living project documentation, accessed 26 September 2026. <https://github.com/eclipse-edc/docs/blob/main/developer/handbook.md>
- [18] Keith Stouffer, Michael Pease, CheeYee Tang, Timothy Zimmerman, Victoria Pillitteri, Suzanne Lightman, Adam Hahn, Stephanie Saravia, Aslam Sherule, Michael Thompson. (2023). *Guide to Operational Technology (OT) Security*. NIST SP 800-82 Revision 3. <https://doi.org/10.6028/NIST.SP.800-82r3>
- [19] Hoeffding, W. (1963). Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301), 13–30. <https://doi.org/10.1080/01621459.1963.10500830>
- [20] Manu Sporny, Ted Thibodeau Jr., Ivan Herman, Gabe Cohen, Michael B. Jones (eds.). (2025). *Verifiable Credentials Data Model v2.0*. W3C Recommendation, 15 May 2025. <https://www.w3.org/TR/2025/REC-vc-data-model-2.0-20250515/>
- [21] Manu Sporny, Amy Guy, Markus Sabadello, Drummond Reed (eds.). (2022). *Decentralized Identifiers (DIDs) v1.0: Core architecture, data model, and representations*. W3C Recommendation, 19 July 2022. <https://www.w3.org/TR/2022/REC-did-core-20220719/>
- [22] O. Terbu, T. Lodderstedt, K. Yasuda, D. Fett, J. Heenan. (2025). *OpenID for Verifiable Presentations 1.0*. OpenID Foundation, Final specification, 9 July 2025. https://openid.net/specs/openid-4-verifiable-presentations-1_0-final.html
- [23] Timothy Lebo, Satya Sahoo, Deborah McGuinness (eds.). (2013). *PROV-O: The PROV Ontology*. W3C Recommendation, 30 April 2013. <https://www.w3.org/TR/2013/REC-prov-o-20130430/>
- [24] Jomi Fred Hübner, Jaime Simão Sichman, Olivier Boissier. (2002). *A Model for the Structural, Functional, and Deontic Specification of Organizations in Multiagent Systems*. In Guilherme Bittencourt and Geber L. Ramalho (eds.), *Advances in Artificial Intelligence: SBIA 2002*. Lecture Notes in Artificial Intelligence, vol. 2507, pp. 118–128. Springer. https://doi.org/10.1007/3-540-36127-8_12
- [25] Ramaswamy Chandramouli, Zack Butcher. (2023). *A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Location Environments*. NIST Special Publication 800-207A, September 2023. <https://doi.org/10.6028/NIST.SP.800-207A>
- [26] SPIFFE Project. (n.d.). *SPIFFE Concepts*. Official documentation, accessed 26 September 2026. <https://spiffe.io/docs/latest/spiffe-about/spiffe-concepts/>

- [27] Elham Tabassi. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1, 26 January 2023. <https://doi.org/10.6028/NIST.AI.100-1>
- [28] Open Source Initiative. (2024). *The Open Source AI Definition 1.0*. Definition, accessed 26 September 2026. <https://opensource.org/ai/open-source-ai-definition>
- [29] Google DeepMind. (2026). *Frontier Safety Framework*. Version 3.1, 17 April 2026. Provider framework. https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/strengthening-our-frontier-safety-framework/frontier-safety-framework_3-1.pdf
- [30] Google Cloud. (2026). *Model Armor overview*. Product documentation, updated 24 September 2026; accessed 26 September 2026. <https://docs.cloud.google.com/model-armor/overview>
- [31] UK AI Security Institute. (n.d.). *Our Research Agenda*. Research programme, accessed 26 September 2026. <https://www.aisi.gov.uk/research-agenda>
- [32] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, Wei Pan. (2023). *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334, Informational, January 2023. <https://www.rfc-editor.org/rfc/rfc9334.html>
- [33] UK Parliament. (2026). *Written question on vulnerabilities in models evaluated by the AI Security Institute*. Question 21384, tabled 28 August 2026; answered by Kanishka Narayan on 7 September 2026. <https://questions-statements.parliament.uk/written-questions/detail/2026-08-28/21384>
- [34] European Commission. (2026). *The enforcement framework of the AI Act*. Explanatory webpage, updated 24 August 2026; accessed 26 September 2026. <https://digital-strategy.ec.europa.eu/en/policies/enforcement-ai-act>
- [35] Kubernetes Authors. (2026). *Pod Lifecycle*. Technical documentation, section on forced Pod termination; accessed 26 September 2026. <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>